

华南理工大学

探 索 性 实 验

报 告

课程名称：	《数据挖掘》
实验名称	景区游览指南的程序性知识挖掘
组号	第___组
组长姓名/学号	组长姓名/学号
组员姓名/学号	组员 1 姓名/学号 组员 2 姓名/学号
学生专业：	软件工程
授课教师：	蔡毅 / 许可 （写自己班的老师）
开课学期：	2026-2027-01

华南理工大学软件学院

2026 年 10 月

地　　点：	B7 后座 楼　　133/231/233
实验日期与时间：	第 9 周周三 9-12 节 第 11 周周三 9-12 节
总评分	分/100 分
实验项目 1 得分	分/100 分
实验项目 2 得分	分/100 分
实验项目 3 得分	分/100 分
实验项目 4 得分	分/100 分
实验项目 5 得分	分/100 分
实验项目 6 得分	分/100 分
实验项目 7 得分	分/100 分
实验总结反思得分	分/100 分
探索性实验总体批改意见：	
批改人	蔡毅、许可

*请不要修改此页

小组信息和组内评分

组号：第　　组

姓名	角色	主要工作和贡献	组内评分
	组长		%
	组员		%
	组员		%

*说明：1、请提交报告时，组长组织组员一起填写好本页，组内评分请根据实际情况，进行合理分配；2、组内人数为N人，组内评分总和为N*100%

共同要求

允许使用等价工具或技术路线。允许 LLM 辅助，但须解释关键逻辑、规则或 Prompt 及验证方法。截图用于证明运行，不能替代源代码、结构化结果和关键中间产物；报告中的结果应能与代码、文件逐一对应。

章节	核心工作 / 报告重点	评分
第 1 章	3 个景区的数据准备、清洗、结构化与质量检查	基础 100 分；附加后封顶 100 分
第 2 章	三类实体定义、识别、位置记录与可视化迭代	100 分
第 3 章	路线解析、时间层级、名称归一化；路线对比为附加题	基础 100 分；附加后封顶 100 分
第 4 章	代词检测、人工标注、消解与效果评估	100 分
第 5 章	条件—建议抽取、分类去重与统计图	100 分
第 6 章	多源融合、图谱模式、构建、展示与质量验证	100 分
第 7 章	LLM 接入、图谱工具、Agent 闭环与多场景测试	100 分
第 8 章	至少 2 个失败现象；局限性与 1 个突破点的研究方案	100 分

《数据挖掘》探索性实验报告
—实验项目 1 景区指南数据采集与预处理

1.1 实验目标与数据准备

这条链子最后的产出是一个能回答"带爸妈去九寨沟玩一天"的 Agent。要让它答得出来，前面得先把散在官网、公众号和小红书里的攻略整理成结构化的东西：哪里是景点、一条路线怎么走、什么条件下该注意什么。所以七个子项目是串起来的，P1 收语料，P2 认实体，P3 抽路线，P4 处理"这里""这条"这类指代，P5 抽条件建议，P6 合进图谱，P7 才拿图谱排行程。本章只做第一步，但后面每一步的输入形状都在这里定下来。

景区挑了三个：广州长隆旅游度假区、珠海横琴长隆国际海洋度假区、九寨沟风景名胜区。前两个是主题乐园，攻略里路线写得密、时间点写得细，适合用来验证路线解析；九寨沟是自然遗产，官方线路和游客实际走法差别大，第 3 章的对比题才有东西可看。三个人各自至少去过其中一个，判断抽取结果对不对的时候不至于全靠猜。

数据全部自采，没有用教师提供的语料。最终 47 篇：景区官网 6 篇、微信公众号文章 9 篇、公众号正文快照 2 篇、小红书游客笔记 30 篇。三个景区都超过"至少 1 条官方路线 + 3 篇游记"的下限，广州 18 篇、珠海 15 篇、九寨沟 14 篇。

表 1-1　三个景区的数据来源与规模

景区	官网路线	公众号文章	正文快照	小红书笔记	合计	清洗后字符
广州长隆	3	5	0	10	18	42441
珠海长隆	2	3	0	10	15	32432
九寨沟	1	1	2	10	14	22304
合计	6	9	2	30	47	97177

官网与公众号：静态页直接抓

官网和公众号都是静态 HTML，requests 加 BeautifulSoup 够用。三个站点的正文容器互不相同：长隆是 div.c--static-flow，九寨沟把正文拆在若干个 div.g-grid 文本块里、要按顺序拼接，公众号是 #js_content。选择器写在 config/sources.json，换站点改配置不动代码。

抓取侧设两条硬约束。同一主机返回 403 或 429 时，停止该主机后续全部请求，不更换 User-Agent 重试；仅对连接超时、502 一类瞬时错误退避重试，最多两次。该策略来自实测：连续五次请求即触发平台限流，账号被限制约半小时。

归档材料与它能证明什么

data/微信公众号与景区官方 下留了 17 条 raw_documents.jsonl 记录、19 个页面存档和一份盘点表。19 个存档里 15 个对应正式文档，另外 4 个是重复抓到或正文为空的，没进数据集；两篇只拿到正文快照的公众号文章单独放在 manual 目录。

这里有个容易被误读的地方要说清楚：crawl_log.csv 记的是归档状态，不是原始 HTTP 请求日志。它只有最终成功或失败，没有每次请求的时间和状态码。所以不能拿它去算访问频率或重试次数，这一点在 1.5 里还会提到，因为它限制了对采集质量能下的结论。

小红书：让 Playwright 接管已登录的 Edge

小红书不登录看不到正文和图片，接口有签名，所以没走 API。做法是用 Playwright 接管本机已登录的 Edge，人先扫码登录一次，脚本复用这个浏览器上下文，在搜索框输关键词、提交、打开笔记、逐张翻图，把浏览器实际收到的响应截下来。人怎么点，脚本就怎么点，只是快一些。

反爬上做了三件事：动作之间加随机延迟（0.8 到 2.5 秒，翻图时更长）；三组关键词交错采样，不同时把同一个景区的笔记连着刷完；只保存浏览器已经加载出来的资源，不额外发请求去补。第二版跑完账号没有触发预警，第一版跑到第 12 篇就被要求验证。

表 1-2　小红书三个批次的采集结果

批次	关键词	目标篇数	实际入库	状态
第一批	广州长隆旅游攻略	10	10	completed
第二批	珠海长隆旅游攻略	10	10	completed
第三批	九寨沟旅游攻略	10	10	completed
合计	—	30	30	—

批次报告里状态是 completed，原始材料打包在 data/小红书爬虫.zip，303,905,941 字节，sha256 为 c4e4b4de…，与 data/小红书爬虫.manifest.json 记录一致。这个包用 Git LFS 存，仓库中常规检出只有 134 字节的指针文件，运行 P1 前须执行 git lfs pull；否则读取压缩包时抛出 BadZipFile。加载环节因此增加了 zip 头与 sha256 校验，不通过即中止。

1.2 清洗方法与实现流程

官网和公众号这一路的清洗拆成三段，一段一个脚本。extractors.py 只负责从 HTML 里取正文块，clean.py 只按规则处理文本，quality_check.py 只做统计和出图。这样拆是因为抽取规则跟着站点变（三个站点的容器结构完全不同），清洗规则基本不变；混在一个函数里，改抽取就要连带重测清洗。

清洗规则九条，R1 到 R9。前六条管字符级噪声（HTML 残留、不可见字符、网址、邮箱、电话、微信号），R7 做全角转半角，R8 删文档内重复行，R9 只标记不删除。每条规则的删除量单独计数写进 quality_stats.json，这样后面能回答"到底删了什么"，而不是笼统说一句"做了清洗"。

表 1-3　清洗规则与逐条删除量（47 篇合计）

编号	处理内容	删除量	说明
R1	HTML 标签残留	0	抽取阶段已剥净，本阶段为兜底
R2	不可见字符	44	零宽空格、软连字符等
R3	网址	1	含裸链接与短链
R4	邮箱	0	—
R5	电话	3	保留景区官方客服号的上下文
R6	微信号	0	防广告引流
R7	全角转半角、空白归一	10779	与 R1–R6 的字符消除合并计入
R8	文档内重复行	769	含截图复读行剔除
R9	异常标记	不删除	写入 quality_flags，供人工复核

表情符号单独计数：606 个。小红书方括号表情按已知名称清理，兼容 R 后缀；未知的方括号标记保留原样，因为分不清那是表情还是作者自己的强调符号。

小红书这一路的差别

小红书没有网页正文可取，能拿到的只有作者写的 caption、笔记里的图片、和已加载的评论。所以清洗前面多一步 OCR：图片送本地 PaddleOCR 服务转文字，再和 caption、评论按顺序拼成正文。清洗阶段完全不重新请求小红书，324 份 OCR 响应全部从本地缓存复用，整轮重跑 19 秒。

官方那 17 篇是既有成果，这次只做了一对一的全角 ASCII 和空格统一，然后按统一正文重建段落和派生统计，不重复删词、去重或重排，历史输入文件一个字节没动。这么处理是为了让 47 篇的统计口径一致，同时不把已经验收过的旧结果再改一遍。

图片 OCR 与作者评论怎么拼

融合顺序是固定的：作者正文 → 按图片顺序的 OCR 文字 → 已采集的作者评论。作者正文不设长度门槛，再短也是作者写的；每张图或每条评论清洗后，汉字、字母、数字合计要严格大于 30 个字符才能进正文，空白、标点、表情、话题都不算。默认只收作者自己的评论，别人的留在辅助资料里。

30 字的阈值由两个反例确定：取 20 时，评论区"谢谢分享""已收藏"等短句大量进入正文；取 50 时漏掉 GZCL-NOTE-07 一条 37 字的外卖地址建议，该条说明园区南大路有便利店可代收外卖。最终取 30，代价是会漏掉部分简短但有效的建议；原始记录全部保留，阈值可调、可重跑。

跨来源去重：截图复读正文

小红书上很多人把文字攻略拍成图发，OCR 回来就是正文的复读。行级去重认不出这种，因为截图里的断行和正文不一样——"我们初一决定初三去长隆野生动物园。初二去超市"在图上可能断成三行。结果是同一段话以碎片形式又进一遍语料，段落数虚高，第 3 章还会在这些碎片上找出根本不存在的"路线"。

补的规则挂在图片分支里。先算作者正文的 4-gram 集合；图片 OCR 的每一行，如果整行被正文包含且不短于 4 字，或者 4-gram 覆盖率不低于 0.8 且行长不短于 8 字，就判为复读剔掉。剔除量计入 dup_line_removed，段落上记 ocr_repeat_lines；一个图片段落全被剔空时，排除原因写"图片文字与作者正文重复"，而不是笼统写"与前文重复"。

0.8 不是拍脑袋定的。先拿真实语料扫了一遍 4-gram 覆盖率的分布：正常内容集中在 0.15 以下，复读集中在 0.9 以上，中间几乎是空的，0.8 正好落在这个空档里。测试写在 tests/test_cross_source.py，六个用例覆盖整行包含、覆盖率刚好达标、长度刚好 8 字、正常内容不误删、空行、跨文档互不影响。

效果是 16 篇内容有变化，共剔 196 行，全库字符从 100222 降到 97177，段落从 4820 降到 4560。最典型的是 ZHCL-NOTE-04，从 4030 字降到 3397 字，剔了 165 行——那篇整篇是十几张长截图，真正的作者正文只有开头几段。

1.3 结构化数据设计

正式输出在 processed/P1 爬虫数据清洗。documents.json 是 47 篇完整文档，含正文、段落、统计和质量标记；documents.csv 只有元数据不含正文，方便用 Excel 打开看清单；paragraphs.csv 逐段一行、带字符起止位置，后面 P2 和 P4 都靠它定位；dataset.xlsx 分文档清单、正文、段落、字段说明四个工作表。

每篇 38 个字段，关键的几个这么设计：content 存最终正文，paragraphs 里每段只存 start/end 指向 content 的偏移，不另存一份文本——这样正文和段落不可能对不上，改正文就必须改偏移，问题当场暴露。raw_char_len 和 clean_char_len 分开记，才谈得上算清洗率。quality_flags 是分号分隔的中文标记，只提示不删数据；真正决定去留的是 is_valid（正文非空、不少于 200 字、且来自详情页）。

表 1-5　文档主表的关键字段

字段	类型	含义
doc_id	str	文档编号，格式为景区码-来源-序号，如 ZHCL-NOTE-04
scenic_code / scenic_area	str	景区缩写与全称
source_platform / source_site	str	来源平台与域名
raw_char_len / clean_char_len	int	清洗前后字符数，用于计算删除率
content	str	最终正文，段落之间以换行分隔
paragraphs	list	每段的 i / start / end / text，偏移指向 content
dup_line_removed	int	重复行剔除数，含截图复读
quality_flags	str	分号分隔的质量提示，不阻断入库
is_valid	int	是否满足长度与来源门槛
has_time_expr	int	正文是否含时间表达，供 P2/P3 参考

辅助资料目录放的是证据不是结果：ocr_results.json 是 324 份 OCR 的原始返回，comments_original.json 是抓到的全部评论（含没进正文的），paragraph_sources.json 记每个段落来自正文、图片还是评论，id_map.json 是 30 篇新笔记的编号映射。P2 生成 mention 时要读 paragraph_sources 判断来源，缺了它直接报错，不会静默降级。

1.4 清洗前后对比与数据质量

47 篇合计 97177 字符、4560 段，最短 402 字、最长 6089 字、中位数 1755 字。字符数按 Python 字符串长度算，含标点和换行，不是分词数；段落按正文换行划分，不等于语义段落——一个语义段落在原文里可能被空行拆成三段。报数的时候把口径写清楚，比数字好看更重要。

分来源看：官网和公众号 17 篇 46236 字 1096 段，小红书 30 篇 51902 字 3528 段。小红书的段落占比（76%）远高于字数占比（53%），因为笔记是一句一换行，官方文档是整段整段写。同一套"段落数"，在两类语料里指的是完全不同的东西，后面 P2 统计段落密度时要记得这一点。

分景区看广州 42441 字、珠海 32432 字、九寨沟 22304 字。广州最多不是因为它篇数最多，而是官网那份两日游写了近两万字，把均值拉上去了。

表 1-6　按景区的结构化规模

景区	篇数	清洗后字符	段数	平均每篇字符
广州长隆	18	42441	1565	2358
珠海长隆	15	32432	1705	2162
九寨沟	14	22304	1290	1593
合计	47	97177	4560	2068

两个具体例子

表 1-7　代表性原文与清洗结果

文档	原文片段	处理结果
GZCL-NOTE-07	#广州长隆[话题]# #长隆带娃攻略[话题]# #亲子游[话题]# …（共 8 个标签独占一行）	整行删除，计入 R8
GZCL-NOTE-06	……时间可能调整，以园区当天公告为准	保留。限定语不能删，否则把作者经历当成现行安排
GZCL-NOTE-06	（作者评论）一日路线整理在图片上，大家可以收藏起来，去玩的时候拿出来跟着走	29 个有效字符，未过 30 门槛，不进正文；原始记录保留
GZCL-NOTE-07	（作者评论）我看我的收货地址是在长隆度假区南大路收费口,外卖小哥一般会给放在美宜佳便利店	37 个有效字符，纳入正文

第一个例子是话题标签，连着七八个标签占整整一行，删掉不影响语义。第二个是限定语，"时间可能调整"在原文里是提醒，删了就把一次具体经历说成了现行开放安排，这类误读到了第 5 章会变成错误的条件建议。

后两个是 30 字门槛的边界。29 字那条内容有用，就因为差一个字符没进正文；37 字那条进了。规则是硬的，边界上必然有误判，所以原始记录都留着——真觉得该收，改阈值重跑就行，不用改代码。

自动质检

tools/quality_p1.py 对 47 篇跑 20 项检查：字段集合与类型、景区和类别映射、doc_id 唯一性、日期格式、来源链接、空值、统计一致性；再逐个值核对 JSON、两个 CSV 和 Excel 是否一致，验证段落编号连续、起止位置能拼回正文、每个新增段落都能在 paragraph_sources 里找到来源。深度检查覆盖 30 篇笔记、324 张图、626 条评论和 47 篇引用到的原始 HTML，图片按 SHA-256 比对。全程离线，不联网。

1.5 问题与分析

跨来源去重对下游的影响可以直接量出来。未做去重时，P3 在 OCR 碎片上生成 42 条路线，其中 20 条被 P6 判为片段剔除；去重后 P3 输出 39 条，片段比例相应下降。问题不在 P6 的判定过严，而在上游把复读内容当成了新文本。

仍有两类问题未处理。其一是 OCR 错字：ZHCL-NOTE-04 末尾的"Ji OSMO podKET3""dJOSMO POCKET 3""JAOSMO POCKET 3"是同一条相机广告的三种识别结果，归一化后分别为 jiosmopocket3、djosmopocket3、jaosmopocket3，行级去重无法识别。这些字符串进入 P2 的候选池后未被采纳，计入 diagnostic_terms。治理需要编辑距离聚类，但该规则容易误伤："诺日朗"与"诺日诺"仅差一字，前者是景点、后者是识别错误。

其二是短文本。JZG-NOTE-07 全文 445 字，其中包含"下午去了树正群海，水很蓝"这类对下游无用的句子，但仍超过 200 字门槛。按长度取值必然存在漏网情况，更细的判据需引入信息密度，本次未实现。

这两类问题会在后续章节反复出现：P2 候选池中的 OCR 残句、P3 中步骤过少的路线、P5 中语义不完整的建议，都与本节描述的数据特征相关。

《数据挖掘》探索性实验报告
—实验项目 2 游览步骤的实体识别

2.1 输入数据与实体定义规范

输入是第 1 章的 47 篇正文，输出是每个景点、每种交通方式、每个时间表达在文中的确切位置。这一步看着简单，实际决定了后面三章能走多远：位置错了，路线就串不起来；角色判错了，"北门"会和"熊猫村"一起被当成景点塞进路线里。

实体分三类。POI 是地点，再分两种角色：attraction 是游客真正去看的东西（熊猫村、长海、白鲸剧场），support 是服务于游览的设施（北门、南门、停车场、换乘中心）。分这一刀是因为路线里只该出现 attraction,但 support 在判断"从哪里进园"的时候又必须有。TRANSPORT 是交通方式(步行、观光车、缆车、地铁、自驾），TIME 是时间表达（下午、第二天、17:00、旺季、半小时）。

三类的时间粒度不一样，不能一把尺子量。TIME 里有"下午"这种粗粒度和"17:00"这种精确值，还有"半小时"这种时长;POI 里有"长隆野生动物世界"这种景区自身名称,它高频出现但不是一个"站点"；TRANSPORT 里"步行"既是交通方式又可能是"步行街"的一部分。这些边界如果不写清楚，词表一匹配就是一堆假阳性。

表 2-1　三类实体的判别口径

类型	子类	判别依据	典型反例
POI	attraction	游客专程前往的景点、场馆、展区、演出场地	"北门"不是景点
POI	support	出入口、停车场、换乘点、餐厅、服务中心	把"南门"排进行程步骤
TRANSPORT	—	连接两个地点的移动方式，可带时长	"步行街"里的"步行"
TIME	时段	上午、中午、下午、傍晚、晚上、清晨	—
TIME	时点	带钟点的表达，如 17:00、九点半	"17:00 左右"的关系词要保留
TIME	日期/时长	第二天、旺季、半小时、两小时	"一天"可能指整日也可能指时长

2.2 识别方法与批量实现

没有用现成的 NER 模型，走的是"词典 + 语境约束"的路子。原因有三个：一是没有标注数据，通用模型对"熊猫村""诺日朗换乘"这种园区内小地名基本认不出；二是后面 P3 要按位置回溯原文，模型给的 span 不一定落在字边界上；三是出了问题要能解释——判错一个词，得说清是哪条规则判的。词典法啰嗦，但每一步都可查。

每个词条除了词本身，还挂一个 guard（语境约束）。guard 决定这个词在什么上下文里才算数。比如"飞"只有在后面跟目的地时才是飞机（判据见 2.3）；"火烈鸟"独立成行不算地点，得出现在箭头链或者路线描述里；"小船"要同时出现"酒店/海洋王国/接驳"和"坐/乘/前往"才算接驳交通。一共十种 guard，写死在 extractor.py 的 classify() 里，每种都有一句可检查的条件和一句拒绝理由。

表 2-2　词表规模与语境约束

景区	词条数	POI 提及	TRANSPORT 提及	TIME 提及	文档数
广州长隆	95	590	146	446	18
珠海长隆	—	556	48	413	15
九寨沟	—	638	239	343	14
合计	—	1784	433	1202	47

匹配用的是最长优先，重叠时按类型分优先级：POI 和 TRANSPORT 冲突时站点优先，避免"长隆野生动物世界南门"被拆成"长隆野生动物世界"+"南门"两个；时间单独处理，完整表达优先于片段。这个顺序写进 policy.json 的 overlap 字段，不是散在代码里。

ID 分两层，这一点后面几章都要用到。entity_id 只由类型、景区码、角色、规范名决定，同一句话挪了位置它也不变；mention_id 由配置哈希、doc_id、起止偏移、类型、角色决定，位置一动它就变。分开的好处是：重新清洗导致偏移全变时，指向"熊猫村"这个实体的引用还稳得住；坏处是凡是记了 mention_id 的地方，重跑一次就要跟着重算。第 5 章会因为这个吃一次亏。

验收用了 39 条定向用例，不追求覆盖率，只钉住容易错的地方：站点不被当成景点、活动不归 POI、跨来源不读上下文、时间关系词保留、错字不自动纠正。每条用例写死了 doc_id 和字符区间，跑完 39 条全过才算通过。这些用例的区间在 P1 改动后要重新锚定（见 1.5），否则会假失败。

2.3 实体输出与出现位置

47 篇一共识别出 3419 个提及，归并成 213 个独立实体。提及数远大于实体数是正常的："北门"在 10 篇里出现 46 次，是同一个实体。报"识别了多少实体"的时候必须说清是哪一个口径，否则 3499 和 213 差 16 倍，读者会以为有一处写错了。

表 2-3　三类实体的提及量与独立实体数

类型	提及次数	占比	每篇平均
POI	1784	52.2%	38.0
TIME	1202	35.2%	25.6
TRANSPORT	433	12.7%	9.2
合计	3419	100%	72.7

每条提及记了这些东西：start 和 end（指向正文的字符偏移）、para_id（属于第几段）、poi_role（attraction 还是 support）、scope（in_area / outside / uncertain）、origin（来自正文、图片 OCR、评论还是历史遗留）。scope 这一项是后面筛景点用的：游记里提到"成都天府广场"这种景区外的地名，识别得出但不能当景点，所以标 outside。现在是 1720 条 in_area、62 条 outside、2 条 uncertain。

origin 的分布也值得看一眼：正文 1136 条、图片 OCR 1067 条、历史遗留 1197 条、评论 99 条。图片 OCR 贡献了近三分之一，这说明小红书那批语料里图片信息量很大——同时也意味着 OCR 的错字会直接变成识别错误，2.4 会细说。

"飞"的判定不依赖固定城市名单，由三条判据共同决定：词表内地名、常见出发地（20 个）、以及行政后缀（市/县/州/机场/景区）。三条命中任一条即视为航空出行，同时排除"飞快""飞过去""飞了起来"这类普通动词用法。本批语料共识别航空类交通提及 433 条；该规则不绑定特定城市，换景区或换出发地仍然适用。

2.4 人工抽查与错误分析

本实验未做全量人工标注，因此不给出 P/R/F1——缺少标注支撑的准确率不具参考价值。比不报更糟。能拿出来的证据有两类：39 条定向验收用例（全部通过），和 302 次被主动拒收的候选。定向用例钉的是容易错的地方，拒收记录说明系统在不确定时怎么处理。这两类加起来只能说明"这些具体判断是对的"，不能外推成整体准确率。

39 条用例的写法是写死 doc_id 加字符区间，断言"这个位置应该有一个 POI"或者"这里不该有"。比如"南门"在正文里出现时要判成 support 而不是 attraction；"白虎跳水"这种动作短语不能出地点；图片 OCR 段不读上一段的上下文（跨来源不传播）；"17:00 左右"的"左右"要留在 time_context 里。每条用例都对应一条真实规则，规则改了用例也要跟着改，改不动说明规则动坏了。

表 2-4　被主动拒收的候选（按出现次数排序）

类型	典型词与次数	为什么拒收	处理方式
出入口/设施	"沟口"36、"南门"28、"北门"11	是 support 不是 attraction	保留角色标记，不进路线步骤
双功能词	"缆车"34、"小火车"30、"空中缆车"8	既指交通方式也指设施名	看前置动词：有"坐/乘"算交通
动物名	"企鹅"36、"火烈鸟"16	活体展区还是动物本身分不清	只在路线描述上下文里才收
OCR 错字	"鹦瞒过山车""熊漏海""山题""海底互动胎"	OCR 误识，不是真实地名	不自动纠正，进 diagnostic_terms
景区自称	"九寨沟"143 次提及	景区名不是站点	识别但标记，解析路线时排掉
抽象词	"水世界"20、"原始森林"5	可能是泛称也可能是具体园区	要求带限定词才收

"沟口"36 次排第一，是个典型。九寨沟的游客常说"到沟口换乘"，沟口确实是入口，但它没有出现在词表里，因为第 1 章的语料里它指的可能是景区大门、也可能是沟口车站。与其猜，不如标成候选留给后面判断——第 3 章解析路线时会看它前后的动词，那时再定也不迟。

OCR 错字那一栏值得单独说。这批语料里，"鹦鹉过山车"被认成"鹦瞒过山车"，"熊猫海"被认成"熊漏海"，"海底互动船"被认成"海底互动胎"，还有"九寨黄龍棍場""珍珠潍站""诺日诺中心站"这种混着繁体、错字的写法。这些词进了 diagnostic_terms 而不是被静默丢掉，是因为它们反映 OCR 的真实错误率——后续如果要拿这批语料训练模型，这些脏数据要能被找出来。

2.5 三类实体词频可视化与迭代

把三类实体的高频提及排出来（图 2-1），能看出几件在表格里看不到的事。

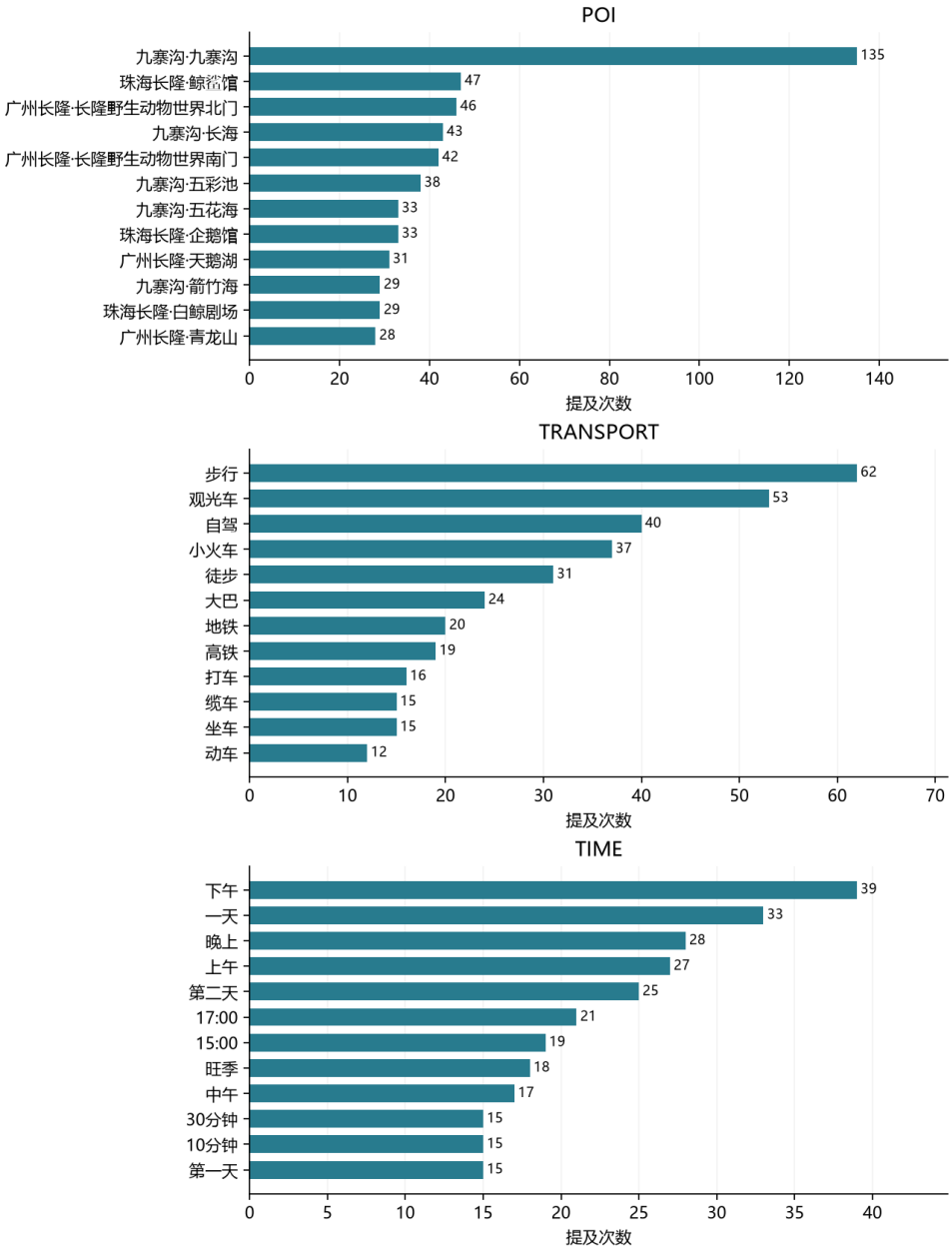


图 2-1　三类实体的高频提及（前 12 位）

POI 第一位是"九寨沟"本身，143 次。这不算噪声——游客写攻略时会反复说"九寨沟怎么玩"，它确实是文中出现最多的地点词，只是它不是一个"站点"，第 3 章解析路线时要排掉。真正有意义的是紧随其后的"天鹅湖"48、"北门"46、"长海"43、"南门"42：出入口的提及量几乎和核心景点持平，说明游客最焦虑的是"从哪个门进、从哪个门出"，这个观察后来直接影响了第 7 章 Agent 的路线描述方式。

TRANSPORT 里"步行"63 次排第一，"观光车"55 次第二。九寨沟的观光车体系是游客讨论最多的，这解释了为什么九寨沟的交通提及（256 条）比广州（146）和珠海（50）加起来还多：沟内换乘是真实存在的决策点，游客会详细写"坐观光车到长海再走下来"。珠海只有 50 条，因为海洋王国园区小，基本靠走，没什么可写。

TIME 里"下午"39 次、"一天"33 次、"晚上"30 次。时点型的精确时间（17:00、15:00）只排到第五第六，说明游记更爱写"下午去熊猫村"而不是"15:20 到熊猫村"。这对第 3 章是个坏消息：路线的时间层级只能靠"上午/下午"这种粗粒度撑，精确到分钟的时刻表很少。

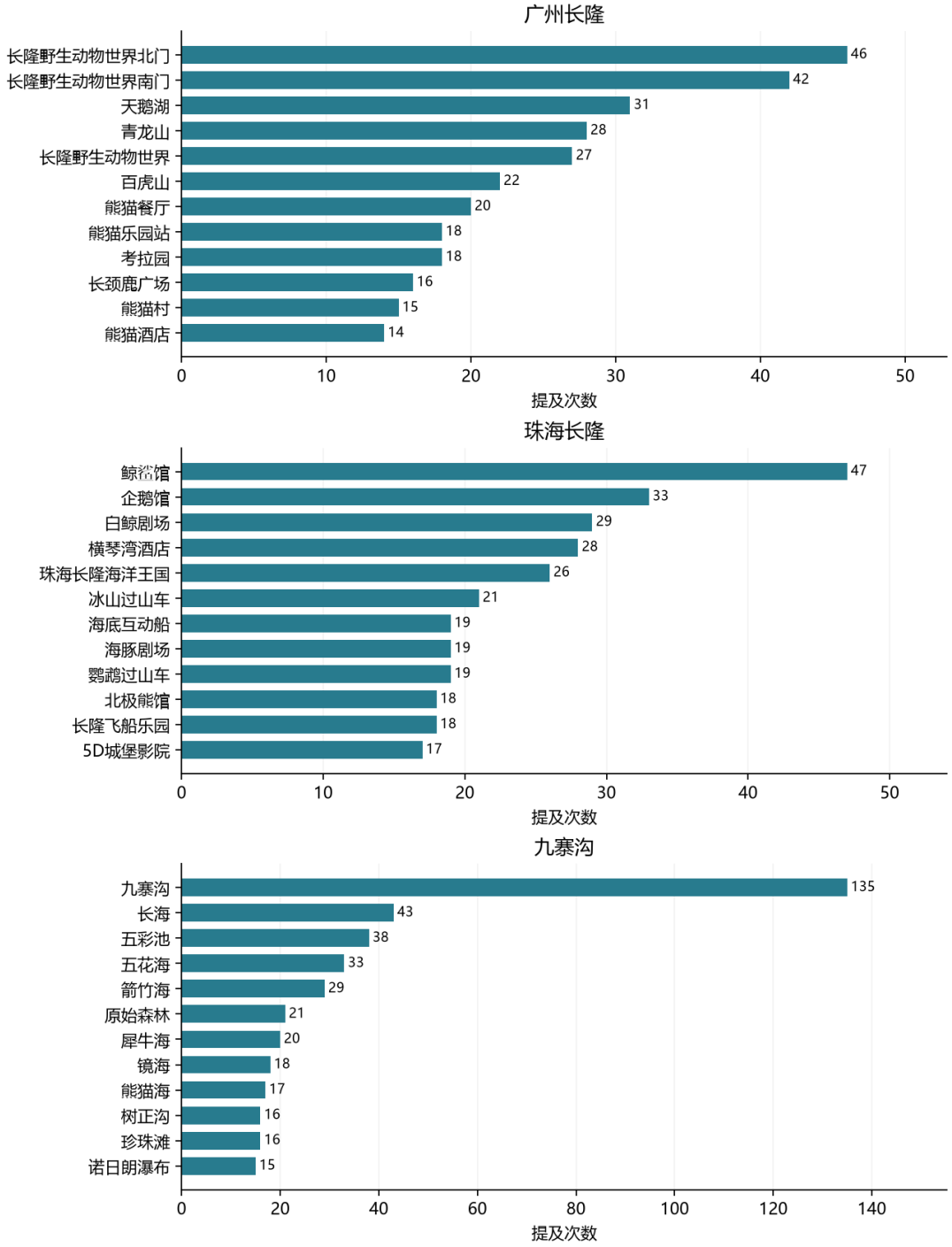


图 2-2　三个景区的 POI 提及分布

分景区看图 2-2，广州和珠海的 POI 提及量接近（593 / 571），九寨沟最高（664）但文档最少（14 篇），每篇密度明显更大。原因是九寨沟的游记会逐个海子点名，"长海、五彩池、五花海、箭竹海、熊猫海、犀牛海"一口气列六七个；主题乐园的游记更爱写"玩了几个项目"这种概括说法，具体馆名反而少。

词表迭代做了三轮，最后落在 9 条补充词上（changes.json）：火烈鸟、白虎山、熊森林、D/E 口、百慕大历险、水母墙、宇宙乐园、海象馆、花海。补的理由各不相同——"白虎山"是原名不该和别名合并，"D/E 口"必须带站点语境才认，"花海"只作为独立列表项才算。这 9 条不是拍脑袋加的，是抽查时发现漏识、确认原文用法之后才写进词表，每条都记了 reason。

迭代过程中有一件事值得记下来：加词比改规则风险大。加"花海"的当天，"五花海"和"火花海"都被切出了"花海"，因为最长匹配在词表内部是按词条顺序比的，新词排在了前面。后来改成先按词长降序排、再按词表顺序，问题才消掉。这类问题不加词不会暴露，加了才暴露，所以每加一个词都要重跑 39 条验收——它们就是用来挡这个的。

《数据挖掘》探索性实验报告
—实验项目 3 游览路线的层级结构挖掘

3.1 路线描述模式与输入范围

47 篇里只有 27 篇真的在描述路线，其余是单点介绍、门票攻略、住宿推荐。判断标准不看关键词，看有没有"可排序的连续站点"：至少两个 POI，且能看出先后。这一步筛错了后面全错——把"必玩清单"当成路线，会解析出一条实际不存在的路径。

书写方式的差异比预想的大。最多的是箭头链，25 条，长这样："北门入园 → 小火车 → 青龙山 → 白虎山 → 长颈鹿广场 → 考拉园 → …"；其次是编号列表 9 条；再往下是按时段分组（"上午去…中午…下午…"）、叙述连接词（"先…然后…接着…最后…"）、按时刻分段（"9:30 到…11:00 到…"）和显式有序列表各一两条。六种格式用同一套解析器处理，靠的是先识别格式再按格式取步骤，而不是写六个解析器。

表 3-1　路线的书写格式分布

格式	含义	条数	解析难点
箭头链（A→B→C）	站点之间用可见分隔符或连接词串起来	24	箭头符号有全角半角、带不带空格等十几种写法
编号列表	站点之间用可见分隔符或连接词串起来	9	序号可能只到 5 然后又从头开始
按时段分组	站点之间用可见分隔符或连接词串起来	2	同一步可能跨两个时段
叙述连接词	站点之间用可见分隔符或连接词串起来	2	"然后"可能只是叙事，不是路线
按时刻分段	站点之间用可见分隔符或连接词串起来	1	时刻和地点不一定在同一句
显式有序列表	站点之间用可见分隔符或连接词串起来	1	列表项可能是并列而非顺序
合计	—	39	—

全文扫描出 417 个候选区间，其中 155 个成为路线主干，258 个被拒。拒掉的主要是三类：跨度太短（只有一两个站点）、跨段落拼不出顺序、以及"必玩清单"式的并列枚举。这些拒收记录都在 candidate_inventory.json 里，每篇文档一条，写清为什么没成路线——"未识别不代表不存在路线"这句话是这一章所有结论的前提。

3.2 统一结构化表示与解析程序

统一表示定成三层：路线（route）→ 步骤（step）→ 引用（ref）。一条路线有 route_id、所属景区、性质（官方推荐还是游客推荐）、粒度（园区级还是园内）、步骤序列；每一步有主景点、伴随的交通引用、时间、以及证据。关键是每一步都挂证据——不是只记"第 3 步是熊猫村"，而是记"第 3 步是熊猫村，依据是 ZHCL-NOTE-04 正文第 152 到 156 字符"。

表 3-2　路线与步骤的字段

层级	字段	说明
route	route_id	由 doc_id 加内容哈希生成，内容不变则不变
route	scenic_code / scope / granularity	景区、作用域、粒度
route	nature	official_recommendation / visitor_recommendation / visitor_unspecified / visitor_actual_claim
route	format	六种书写格式之一
step	poi_refs	主景点与伴随设施，带 poi_role 和 entity_id
step	transport_refs	该步用到的交通方式
step	time	时段、钟点、第几天，附证据
step	evidence	doc_id + 起止偏移 + 原文片段

三个脚本分工：discovery.py 找候选（哪些段落像路线）、parser.py 按格式取步骤、pipeline.py 把步骤里的地名关联回第 2 章的实体和时间。分开的好处是改格式解析不影响实体关联，P2 重跑之后只要重跑 pipeline 这一步。

证据挂到步骤上还有一个副作用：可以直接量出精度上限。312 步里有 309 步的 evidence 区间能拼回原文，拼不回去的会被判失败——这就要求 P1 的段落偏移必须和后面对得上，第 1 章那条"正文和段落不可能对不上"的设计就是为这个准备的。

3.3 时间层级构建

时间做了三层：时段（清晨/上午/中午/下午/傍晚/晚上）、钟点（17:00）、天数（第二天）。三层不是并列的，是包含关系——一天里分时段，时段里可能有钟点，多天才有天数。所以时间层级图是按"天 → 时段"画的，钟点作为时段内的标注。

做完发现最多的不是任何一个时段，是"未说明"：309 步里 200 步没有时间信息，占 65%。这跟第 2 章的观察对得上——游记爱写"下午去熊猫村"，但更多时候连"下午"都不写，直接"接着去了熊猫村"。时间层级只能覆盖一部分步骤，这是语料本身的性质，不是解析漏了。

表 3-3　步骤的时段分布

时段	步数	占比
未说明	200	64.7%
下午	48	15.5%
上午	35	11.3%
中午	15	4.9%
傍晚	6	1.9%
晚上	4	1.3%
清晨	1	0.3%

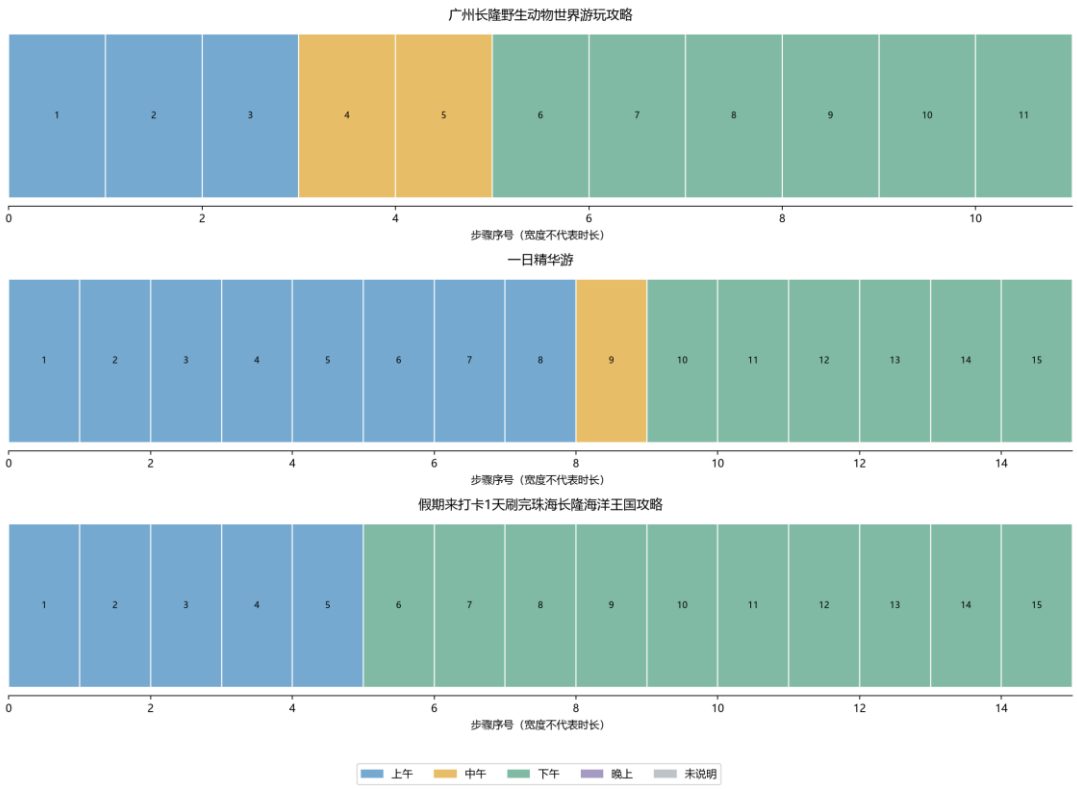


图 3-1　三条路线的时段分段（宽度不代表时长）

图里三条路线，颜色是时段。第一条 11 步，上午 3 步、中午 2 步、下午 6 步；第二条 15 步，只有第 9 步标了中午，其余全是上午和下午；第三条 15 步，前 5 步上午、后 10 步下午。能看出的一件事是：时段是"块状"的，很少交替。游客不会上午下午上午来回跑，都是上午集中走一片、下午走另一片。这对第 7 章排行程有用——裁剪的时候不该打乱这个块结构。

精确到分钟的钟点不多：312 步里 67 步带时钟文本，其中能解析出具体分钟的更少。所以"07:00—17:35"这种官方时刻表在语料里是特例，不能当成通用能力来宣传。九寨沟那几条带时刻的路线全出自官网，游客笔记基本不写钟点。

3.4 名称归一化与稳健性

名称归一化只做一件事：把同一个地方的不同写法并到一个 entity_id 上。第 2 章已经做好了"长隆野生动物世界"和"广州长隆野生动物世界"并成一个，"北门"和"长隆野生动物世界北门"并成一个。这一章在路线里直接引用 entity_id，不再自己判重。

两条不做：不确定的别名不合并，疑似错字不自动纠正。"白虎山"和"百虎山"在原文里都出现过，但只有一个是真的园区名，程序判断不了，就都保留原样、其中没进词表的标成"未归一"。16 组对比里只有 3 处"未归一"，说明归一化覆盖得还行，也说明剩下这 3 处确实难判。

表 3-4　归一化与不归一化的例子

原文写法	处理	理由
长隆野生动物世界 / 广州长隆野生动物世界	并为一个 entity_id	同一园区，前者是简称
北门 / 长隆野生动物世界北门	并为一个 entity_id	同一设施，前置了园区名
白虎山 / 百虎山	不合并	疑似错字，无法确定哪个是原文本意
沟口	保留为候选，不进路线	可能是大门也可能是车站
诺日朗 / 诺日朗换乘 / 诺日诺中心站	前两个合并，第三个判错字	OCR 把"朗"认成"诺"

这套规则在跨章的时候有个约束：归一化必须以 entity_id 为锚，不能以字符串为锚。第 5 章的条件建议和第 6 章的图谱都靠 entity_id 连起来，只要锚不变，中间换多少次写法都不影响。反过来说，一旦某条规则改变了 entity_id 的生成方式（比如把 poi_role 纳入哈希），整条链都要重跑，这一点在第 6 章的级联约束里有对应说明。

3.5 结构结果验证

最终 39 条路线、309 步，平均每条 7.9 步，最长 28 步、最短 2 步。分景区：广州 19 条（官网 3 篇写得多，路线也长）、九寨沟 11 条、珠海 9 条。

表 3-5　分景区的路线与步骤

景区	路线数	性质分布	平均步数
广州长隆	19	3 / 16	5.5
九寨沟	11	2 / 9	10.6
珠海长隆	10	1 / 9	9.0
合计	39	6 / 33	7.9

验证做了几项：每条路线的步骤序号连续、每步的 evidence 能拼回原文且位置正确、路线内的步骤不重复引用同一段原文、路线之间的区间不重叠（同一段文字不能既算 A 线又算 B 线）。最后一项针对区间重叠：同一段文字不得同时属于两条路线，否则 P6 合并时会重复计数（早期版本存在该问题，已在区间分配阶段消除）。

还有 14 条路线只有 3 步。这不算错，原文就写了三个地方（"南门进，看熊猫，北门出"），硬凑成更长的路线反而是编造。第 7 章的 Agent 会给这类路线打个"只到园区级"的标记，排两天行程时提醒一句，而不是硬当作两日游方案输出。

7 条诊断记录，主要是"候选跨段落"和"步骤里的地名没对上实体"两类。跨段落的那种，原文是"上午先去了 A、B，下午 C、D"，中间隔了一段讲吃饭，程序不敢跨过去拼，这种保守是有意的：宁可少拼一条，不要拼错一条。

3.6 官方与游客实际路线对比【附加题】

附加题要回答的是"官方推荐路线和游客实际走法差在哪"。做法不是人工找对照，是自动配对：同一景区、同一粒度、且至少有两个共同景点的官方路线和游客路线配成一对，然后算三个指标——官方景点覆盖率（官方路线里的景点有多少出现在游客路线里）、Jaccard 系数（两者景点集合的交并比）、以及共同景点的顺序相似度（LCS 归一化）。配出 16 对。

结果比预想的平淡：官方覆盖率平均 0.39，最高 0.80、最低 0.13；Jaccard 平均 0.31，最高 0.60、最低 0.12。也就是说，官方路线里平均只有三成多的景点会被游客真的走到。拆开看更清楚：平均每对共同景点 5.2 个，官方独有 9.2 个，游客独有只有 2.4 个。官方路线长、列得多，游客只走其中一段，偶尔加一两个官方没写的点。

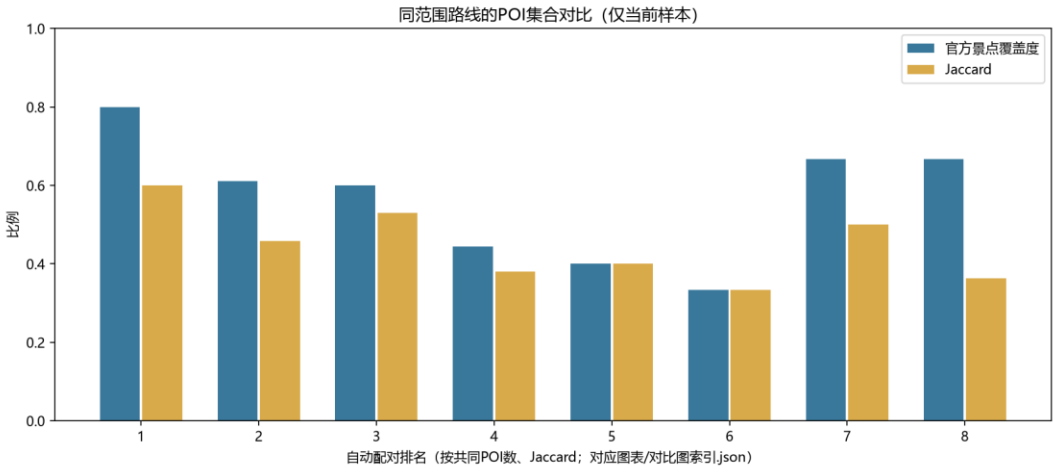


图 3-2　同范围路线的 POI 集合对比（自动配对，仅当前样本）

表 3-6　配对结果的分布

指标	平均	最低	最高	说明
官方景点覆盖率	0.39	0.13	0.80	官方列的景点被游客走到的比例
Jaccard 系数	0.31	0.12	0.60	两者集合的交并比
共同景点数	5.2	2	12	两个都提到的景点
官方独有点	9.2	2	15	官方有、游客没写
游客独有点	2.4	0	6	游客提到、官方没列

覆盖率最低的那一对（0.13）是广州官网两日游和一篇一日游笔记配上的。官方那份列了 15 个点，笔记只走了其中 2 个。这不是游客"不听话"，是两天和一天的容量本来就不一样——配对规则只看"至少两个共同景点"，没看行程天数，这是个已知的缺陷。若按天数再分一层，该指标才具备可比性，本次未实现。

顺序相似度只对 14 对适用，另外 2 对共同景点少于 2 个，算不了 LCS，标成 not_applicable。14 对里有几对的相似度在 0.75 以上，说明大方向一致（都是从北门一路往南），差异主要在中段——官方让你把每个馆都看一遍，游客会跳过几个直奔下一个。

结论要说清楚边界：这批"游客路线"是从文字里解析出来的，不代表游客真的按这个顺序走了。有人写"看了熊猫村、考拉园"只是列举，不是路线。所以这里的差异只能说明文本层面的差异，不能推成"游客偏好"。原文里没有的东西，报告里也不该有。

《数据挖掘》探索性实验报告
—实验项目 4 程序性知识的共指消解

4.1 代词检测与待消解样本

代词检测走两条路。一条是正则，把“这/这里/这个/那/他/它/它们/她/她们/这些/那些/这种”这类触发形式在全篇扫一遍；另一条不检测，直接用第 2 章已经识别出来的实体提及。两条路的结果并成一张候选表，最后得到 545 条待消解记录，落在 41 篇正文里（剩下 6 篇从头到尾没有出现指代，这些文档不参与本章，但分词照样做了）。

触发形式的分布很集中。“这”一个字出现了 159 次，“这里”91 次，两个加起来占了一半多。这和游记的写法有关：写完一段景点介绍，接着用“这里”“这个”把话头接下去。人称代词少得多，“他”42 次，“它”25 次，“她们”只有 9 次。

表 4-1　触发形式分布（共 545 条）

触发形式	次数	常见语境
这	159	指正在讲的这个景点或这件事
这里	91	指当前段落所在的地点
这个	61	指刚提到的单个对象
那	48	回指更早的内容，常带对比
他	42	指游客本人或同行的人
它	25	指动物、场馆，偶尔指一个项目
它们	21	指一群动物
她	12	指女性游客
这些	11	指刚列举的一组
（无触发）	9	没有独立触发区间，整条记录按事件处理

区间长度上有个细节：235 条指代只有一个字，310 条两个字。单字的“这”麻烦最大——它后面可能跟量词（“这条线路”），也可能只是个指示词（“这时候”）。本版的处理是把“这个/这种/这些”这类两字形式整体当触发区间，单字“这”只取它自己，后面跟的完整名词另存一列

reference_phrase（“这条线路”“这种安排”），这样既不会把量词算进指代，也不会丢掉语义。这条规则是统一定的，没有照着标注答案调边界，代价是少数样本的区间和人工标注差一两个字，4.5 节里有一条就是这么错的。

候选池的规模差异很大：平均每条记录 7 个候选，但有 76 条记录一个候选都没有。这 76 条不是程序出错，是那句话里同句和前两句确实没有名词性成分（“这样可以省很多时间”），这种直接判未消解，不进打分。反过来，有的记录候选堆到十几个，因为一段话里连着出现七八个景点名。

4.2 人工标注真实先行词

人工标注只在 5 篇上做，共 51 行，因为逐字核对整篇太慢，而这一章要的是“规则在真实指代上错在哪”，不是覆盖全量。5 篇里广州两篇、珠海一篇、九寨沟两篇，长的 4000 字、短的 800 字。

这 51 行里有 2 行是两个来源：自动扫描命中的 35 行，以及人工通读全文后补进来的 16 行。补的那些是自动扫描漏掉的——比如“这里”藏在长句中间，或者干脆是“这儿”这种词表里没有的写法。两类都标 sample_origin，统计时可以分开看。

表 4-2　标注构成

维度	取值	行数	说明
是否待消解	is_target=yes	41	需要给出先行词或判定无先行词
是否待消解	is_target=no	10	扫描误命中，不是指代，不进分母
标注状态	antecedent_found	34	有明确先行词，且能指出原文区间
标注状态	non_entity_reference	5	“在这里感谢”这类话语标记
标注状态	no_clear_antecedent	2	意思懂，但指不出具体是哪个
来源	auto_detected / manual_added	35 / 16	自动命中 / 人工补入

标注的字段不多，但每个偏移都要能回切原文：reference 是指代本身，sentence 是它所在的整句，context 是指代前后各取一段（前 65 字、后 85 字），antecedent 是先行词。四个区间都必须满足 content[start:end] 等于标注文本，差一个字即判定失效。正文一旦重新清洗，全部偏移需要按内容重新定位：先按整句在新正文中检索，再按句内偏移还原指代；先行词按片段检索，多处命中时取位于指代之前且最近的一处；上下文窗口按"前 65 字、后 85 字"重算。重定位只调整位置，不修改标注内容，原文片段被删除的直接报错。

真正难标的是两类。一类是事件先行词：原文列了两段路线清单，后面说“完全可以复制这条线路”，先行词是那两个区间，一个记录要挂两个先行区间（就是标出来的那 1 条多先行词样本）。另一类是话语标记：“在这里感谢写攻略的朋友”，这里的“这里”不是地点，没有实体可以指，只能标

成 non_entity_reference，且不算错。这两类占了错误分析里的大部分篇幅，也是后面 4.5 节要分开统计的原因。

4.3 消解方法与实现

候选先行词来自三个通道。第一通道是第 2 章的实体提及，直接入池，因此“澳洲森林”“黑颈天鹅”这类复合专名不会被分词切成两块。第二通道是独立分词：jieba 0.42.1，HMM 关掉，把本篇出现过的景点全名注入自定义词典（75 个词），全篇 56471 个词元，取名词性词性（n/ng/ns/nz/nr）作为补充候选，相邻的修饰词加中心词再拼一次，让只出现一次的复合专名也有机会。第三通道是事件：以动词、箭头链、编号结构为锚，生成可以回切原文的事件区间，最长 600 字，超了就记 generation_limits，不截断冒充完整事件。

打分是三项加权：类型适配 0.45、距离 0.35、层级 0.20。距离按字符算，用 1/(1+d/12) 衰减，所以隔 12 个字掉一半分；层级同句给 1、上一句给 0.5、再往前给 0。类型适配看指代被判成什么：判成地点就只给地点和设施加分，判成动物就压掉地点分。最后过两道闸——最高分要到 0.65，且和次高分差至少 0.10，否则判歧义、不输出。两个阈值的作用是拦下“距离近但类型不符”的候选，取值来自开发集，未做独立标定。

候选不是想搜多远就搜多远。实体只搜同句和前两句，字符窗口 180；本地没有类型合适的候选时，允许回看紧邻的上一段，但同来源才算数；事件最多回看 1200 字。被这些约束挡掉的候选有 36464 次，挡得最多的是跨段落（24909）、主题或行程切换（3894）、正文与图片评论混接（7307）。这套约束是保守的：宁可判未消解，也不跨段硬连。


```
{"reference": {"start": 137, "end": 139, "text": "这里", "type": "location"},
 "candidates": [{"text": "鲸鲨馆", "semantic_type": "POI", "dist": 41, "score": 0.78},
                {"text": "海底互动船", "semantic_type": "POI", "dist": 63, "score": 0.51}],
 "status": "resolved", "antecedent": {"text": "鲸鲨馆", "start": 96, "end": 99},
 "score": 0.78, "runner_up_score": 0.51, "rule_ids": ["P2_ENTITY", "LOCATION_MATCH"]}
```


每条记录都把候选、分数、次高分、命中的规则一起存下来。这么做是为了区分两种失败：候选里根本没有正确先行词（候选生成的问题），还是正确先行词在候选里但排第二（排序的问题）。改法完全不同，混在一起看就只会得出“效果不好”这种没用的结论。

4.4 消解结果与输出

545 条记录里，给出关联的 76 条，判未消解的 394 条，排掉的 71 条（词表判定不是指代），还有 4 条是话语标记。关联率 13.9%，偏低源于保守策略：第 6 章用这些关联连图谱，错连的代价高于漏连。

表 4-3　未消解的原因分布

原因码	条数	含义
type_mismatch	247	候选类型与指代类型不符
lexical_non_reference	71	词表判定不是指代
type_distance_rule	64	类型加分后仍不够
no_candidate	50	同句和前两句没有名词
event_candidate_missing	47	事件候选没生成出来
ambiguous	27	两个候选分差不到 0.10
low_score	23	最高分没过 0.65
event_structure_rule	12	事件结构不完整
discourse_expression	4	话语标记，没有实体先行词

类型不符是最大头（247 条）。典型情况是“这里”后面最近的候选是“排队”“拍照”这类动作名词，或者“这个地方”旁边写着“人很多”，程序拿不到地点类型的候选，就判未消解。排第二的是“候选类型对但分数不够”，多数是距离太远——正确的景点名在前面三四句，中间隔了一整段吐槽。

表 4-4　分景区的消解情况

景区	记录数	关联	未消解	排除	关联率
广州长隆	273	41	196	33	15.0%
珠海长隆	173	21	127	25	12.1%
九寨沟	99	14	71	13	14.1%

九寨沟的关联率最低（14.1%），原因不在算法。九寨沟那几篇写的是“从诺日朗坐车到长海，再走栈道去五彩池”，指代多为“这里”“那边”，但前面铺的是交通和路线，实体密度低；而广州的游记一句一个馆名，“这里”很容易找到地点候选。这是语料性质，不是阈值设错了——把阈值降到 0.55 能把九寨沟的关联数提上去，但同时会多出一批错连，4.5 节的基线就是这种情况。

输出三份：coreferences.json 是正式产物，带候选、分数和规则号；tokenization.json 存 47 篇的分词结果和词典哈希，方便复盘“为什么这个词没被当候选”；outputs/P4 下的检查表和附注文本只给人看，不参与下游。

4.5 人工核验与评价

评价只算 51 行人工确认过的标注，其中 41 行是待消解的目标。匹配规则是严格的：指代区间和先行词区间都要完全一致，多个先行词要集合完全相等，差一个字算错。不设“部分正确”，也不给边界重叠算分——那样数字会好看，但没意义。

表 4-5　规则系统与基线的对比（同一批标注、同一套区间匹配）

指标	规则系统	基线（取最近实体）	说明
输出关联数	14	23	基线敢连，规则敢不连
关联正确数	11	8	严格区间一致才算对
关联正确率	78.6%	34.8%	对已输出关联的精确度
目标覆盖率	34.1%	56.1%	41 个目标里有多少拿到了关联
检测召回	95.1%	95.1%	能认出“这里”是在指代，两套一样
状态判断准确	58.8%	66.7%	消解/未消解/排除判对的比例

规则系统输出 14 条关联，对 11 条，错 3 条；基线输出 23 条，对 8 条。基线的覆盖率反而更高（56.1% 对 34.1%），状态判断也更准（66.7% 对 58.8%），因为它几乎总是硬指一个最近的实体，判“未消解”的次数少。这就是取舍：基线比规则多连了 9 条，代价是它自己的 23 条里有 15 条是错的；规则连得少，但已连部分正确率更高，两类指标不存在单向优劣。

表 4-6　按先行词类型的成功率（单先行词 33 个）

类型	目标数	正确	成功率	暴露的问题
事件	14	4	29%	事件区间太长，边界定不住
地点	11	6	55%	本类最好，仍有 5 例被距离拖累
动物	3	1	33%	指代笼统（“它们”指一群），单个动物名对不上
设施	3	0	0%	设施名和地点名混在一起，类型判不出来
其他	2	0	0%	样本少，两类都没对

三条错误各有具体成因。P4A-011 的“这样”，人工标的是“穿大口袋的衣服……就进去了”这一小句，程序把整句（连前面的“你可以背包进去”）都圈进去了，区间过宽；P4A-031 的“这个”人工指的是“容易忽略的海底互动船”，程序连到了同一段的“鲸鲨馆”，属于同名景点密集时的排序错误；P4A-038 的“这”两侧文本完全一样，都是“五彩池”，只是起点差了两个字，严格匹配下算错——该例属于边界规则的代价，而非理解错误。

与 V1 版本相比，关联数由 5 条增至 14 条，正确数由 4 条增至 11 条，增量主要来自事件类（4 条正确）与复合专名类（“澳洲森林”“黑颈天鹅”等）。多先行词那 1 条依然失败：两个区间要一起命中才算对，程序只给出其中一个。

这些数字不能当模型准确率用，理由有三条，都得写在明面上。一是样本只有 5 篇、41 个目标，错一条就是三个百分点；二是这 5 篇在规则开发时反复看过，属于“已见样本”，不是留出集；三是阈值 0.65、分差 0.10 和几条词表都是照着这批错误改的，换一批游记重测，正确率大概率会掉。可信的泛化指标需要另标 10 篇以上未参与调参的样本，本次实验未覆盖。

《数据挖掘》探索性实验报告

一实验项目 5 条件性游览建议的抽取

5.1 输入与抽取规则设计

这一章要抽的不是"景点在哪"，而是"什么条件下该做什么"：下雨带雨衣、儿童 1 米到 1.5 米买优惠票、自驾要另买自驾车票。这类信息在游记里散着，句式也散，规则写不全，所以这一章交给模型抽取，但只让它做一件事——把一段原文翻成若干条结构化记录，别的一概不管。

输入切法沿用包（package）的概念：原计划 2042 个包（段落包 2003 个、路线包 39 个）；其中 1 个路线包超预算，按段落切成 4 片重跑，因此实际执行 2045 个包。每个包是一到几个完整段落加上该段的来源信息。包不能太大——模型要在一次调用里读完原文再一次性交出结果，超了就只能丢内容；也不能太小，否则"如果下雨就不建议去"这种跨句条件会被切断。

表 5-1　抽取工具与产出

工具	产出	关键字段	本次条数
submit_poi_facts	景点事实	attribute / value_text / modality / condition	475
submit_conditional_advice	条件与建议	condition / consequence_text / force / targets	1327
submit_route	路线候选	steps / order / evidence	151 条新路线
submit_transport_leg	交通路段	from / to / transport / duration	105

规则层只做两件事：定字段，定边界。字段是死的——条件必须有 text 和 clauses，条款要落到受控类别上（日期时段、门票、人群、天气、拥挤、体力、进出园、服务可用、时间预算、其他）；建议必须有 consequence_text 和 force（推荐、必需、禁止、不建议、可选、只是描述、未表态）；每条记录都必须挂回一个输入包。边界是活的——模型自己判断一句话切成几条，但每条都要能追到那段原文。

引句定位不由模型完成：模型只负责把原文抄进 consequence_text / value_text，偏移由导出程序在包内检索。先按原字符串精确匹配，标点或全半角不一致时退到规范化串，唯一命中记为 span 并写下起止位置，多处命中或找不到则退回 package，不做模糊猜测。本次 1622 条带引文的观察中 1448 条定位到句（89.3%），共 1791 条引文；未定位的 174 条多为改写过的短句与跨段拼接，保留包级来源。

让模型报字符偏移是另一种做法，但模型算偏移经常出错，错了以后整条记录要么报废、要么被迫相信一个错位置。把定位交给确定性程序，模型只承担抄录，既避免偏移算错，也让伪造引文的成本变高：抄错的话在原文里找不到，只能退回包级。

5.2 批量抽取与复杂表达处理

共提交 2045 个包，全部返回 COMPLETED。合计得到 3516 条观察，从中整理出景点事实 475 条、条件与建议 1327 条、交通路段 105 条。观察数比记录数多，因为一次提交可以带多条记录——一段话里写了三个景点的开放时间，就是一条观察、三条事实。

表 5-2　批量运行开销

项目	数值	说明
输入包	2042	路线包、段落包与补跑分片之和
请求数	4,314	平均每包 2.1 次（含重试）
输入 tokens	34,848,420	其中缓存命中 33,842,099（97.1%）
未命中计费 tokens	1,006,321	按原价计价的输入部分
输出 tokens	846,557	结构化 JSON
单次最大提示 tokens	14,368	最长的一个包（整条路线）
模型	deepseek-flash	温度 0，强制 JSON 输出

开销集中在输入侧，但计价不能只看输入总数：4314 次请求共 34,848,420 输入 token，其中 33,842,099 命中前缀缓存（97.1%），按未命中计价的只有 1,006,321；输出 846,557 token，平均每次 196。缓存命中来自固定的系统提示与工具 schema——每个包只在末尾追加自己的正文，同一批次里除首次请求外，前缀部分几乎全部复用。

批处理按包复用：检查点记下包指纹（包内容、模型配置、实现哈希、提示词资产），重跑时指纹一致的包直接拷结果、不再请求，不一致的照常发起。上游只改少量文档时，开销按变更面而不是全量计。

一个包含有整条九寨沟官方两日路线的包触发了上下文预算上限。该包按段落切为四片补跑，校验原文覆盖与步骤集合不变后合并回批次，因此实际执行 2045 个包（原计划 2042，失败 1 个、补跑 4 片）。

复杂表达主要三类。第一类是多条件："晴天且非节假日才建议走西线"这种，clauses 会挂两条、logic 标 all；本次 1327 条条件里有 39 条是两个以上子句，38 条用了 all 或 any。第二类是条件藏在动词里："走不动就别走栈道"，没有"如果"两个字，模型要把"走不动"识别成体力条件，这类最容易漏。第三类是口语写法："带娃""小盆友""特种兵"，人群字段是自由文本，不做归一，留给后面的统计算粗分类。

还有 58 条判成"条件不完整"——模型读到了一半条件，后半句被段落切断，或者原文本身就写了一半。这些没有丢掉，也没有硬补，单独标成 incomplete_condition 留在库里，第 7 章的 Agent 不会把它们当建议用。

5.3 映射表、分类与去重

映射表分两层。第一层是目标映射：条件建议要挂到具体对象上（哪个景点、哪条路、哪种交通），挂不上对象的留 target_id 为空，标成"景区级未定位"，不硬塞给某个景点。第二层是类别映射：604 条条件子句按受控类别落位，日期时段最多（176），其次是"其他"（176）——这一类里多是"人多的时候""第一次来"这种没法归类的说法。

表 5-3　条件子句的类别分布

类别	子句数	典型原文
日期时段	176	“4 月 1 号已开放”“旺季”
其他	176	“人多的时候”“第一次来”
人群	90	“带娃”“身高 1 米到 1.5 米”
门票	30	“需提前一天 23:59 前预订”
进出园	29	“如需自驾须持自驾车票”
天气	25	“下雨天”“晴天”
时间预算	24	“时间充足的”
拥挤	23	“假期人很多”
服务可用	19	“园区内租借的推车”
体力	12	“走不动太久”“恐高”

force 字段记录说话人的确定程度，按"语气"理解：1327 条里"推荐"最多（839），"必需"只有 46 条，"禁止"和"不建议"加起来 77 条。这个分布符合游记的写法——大部分是"建议你怎么样"，真正硬性的规定（禁止带食物、必须提前订票）很少，而且基本都出自官网那几篇。

去重按字面执行：仅当目标与条件相同、且规范化文本完全一致时才合并，结果 1327 条规则并成 1787 个条目，只有 1 个条目来自两条以上的规则。原因不是流程有问题，是语料里几乎没人用一模一样的措辞说同一件事："需要雨衣"和"记得带雨衣"是一回事，但按字面不算重复。近义合并需要语义判断，本次未实现，仅在 outputs/P5/校验/去重与近重复候选.json 中保留候选清单。

目标映射的粒度也记了一笔：3516 条观察里，引用范围落在段内提及的 2931 条、落到景区实体表的 1034 条、落到整个输入包的 464 条。最后那 464 条只能说明"这篇文档提过"，不能说明"这个景点适用"，后面 P7 取建议时会优先用前两类。

5.4 人工校验与误差分析

人工核对按陈述类型分层抽取 10 条：六种类型各取一条，另加 4 条随机，随机种子固定，结果可复现。逐条回原文比对三项：内容是否一致、条件是否遗漏、语气是否夸大。明细写在 outputs/P5/校验/人工核对 10 条.csv，含规则 ID 与原文片段，可逐条复核。

表 5-4　10 条分层核对

类型	抽取结果	原文	判断
条件建议	排了 40 米的 → 不必坐 25 米的	雨林升降塔 40 米(排了 40 米的就没必要坐 25 米的了)	合格
条件事实	跟秋游大军错开 → 优速通没用上	海洋王国去其实我们都没等过(跟秋游大军错开玩了)	合格：条件是事后归因
一般建议	住熊猫酒店结束后步行回	住熊猫酒店结束后直接步行回。	合格
条件限制	刚装修完 → 没选企鹅酒店	没有选择企鹅的原因很简单,刚装修完没多久	不合格：个人取舍写成通用限制
条件不完整	20:33 黄龙九寨（或 20:40 松潘）→ 空	20:33 黄龙九寨(或者 20:40 松潘)C5758→22:20 成都东	合格：正确标为不完整
一般限制	不可喂食	可开车窗,不可喂食,中途不可下车	合格：丢了"自驾"场景限定
一般建议	10:30 看百鸟飞歌	10:30 准时看百鸟飞歌(超震撼)	合格
条件事实	秋游大军十点来、三四点走 → 同文	不过她们一般十点来,三四点就走了	不合格：条件与结论同文
一般建议	观光车最省心	观光车最省心~	合格：周边有 OCR 噪声
条件建议	带 1 岁以内宝宝 → 选企鹅酒店	Tips5:如果带着 1 岁以内的宝宝来长隆还是建议选择企鹅的	合格

10 条中 8 条与原文一致，2 条不合格，失败方式不同。一条是类型判断越界：作者写"没选企鹅是因为刚装修完"，这是个人取舍的因果解释，被抽成 force=avoid 的通用限制，语气被抬高。另一条是结构冗余：条件与结论来自同一句话（"秋游大军十点来、三四点走"），展开后成了"如果 X 则 X"，没有新增信息。两类都说明问题不在抄录，而在判定：什么算条件、什么算建议，规则层给不出更细的约束。

另有 58 条被标为 incomplete_condition，多为段落截断或原文本身只写了一半。这类没有硬补，单独留在库里，第 7 章的 Agent 不把它们当建议用。抽查中 1 条（第 5 条）属于此类，判定正确，说明"条件完整度"字段在截断场景下可用，但不能反过来当作内容正确的证据。

核对过程还暴露三处细节问题。一是标点被规范化（全角逗号变半角），按原文精确匹配会失败，核对脚本要先做标点归一。二是口语词被删（"就不建议"→"不建议"），读起来更整齐，但严格说改动了原文。三是时态类事实（"4 月 1 号已开放"）被抽成无条件事实，库里统一标了 current_validity_verified=false，使用时需复核年份。

10 条样本不足全量的百分之一，不能据此推出整体准确率。若要给出准确率，需按类型分层抽取 100 条以上、由两人独立判定并计算一致性，本次实验未覆盖，因此本节只报告个案层面的核对结果。

5.5 分类统计与可视化

六种陈述类型在三个景区都以"一般建议"为主：广州 272 条、珠海 226 条、九寨沟 231 条，分别占各景区规则的 50%、54%、63%。条件类（条件建议与条件事实合计）在珠海占比最高，与海洋王国攻略的写法有关：身高限制、天气影响、项目时长多以条件句给出。

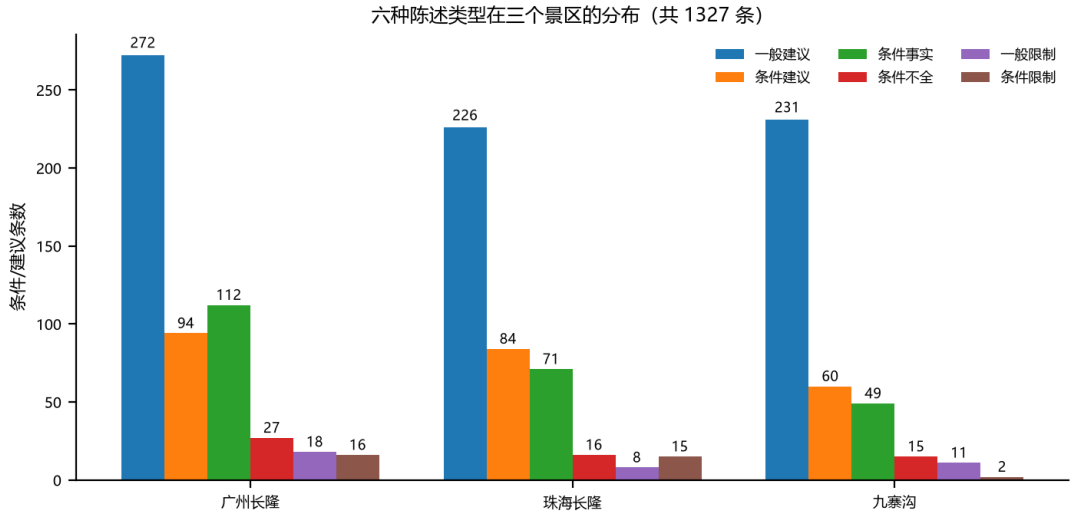


图 5-1　六种陈述类型在三个景区的分布

条件子句的类别分布见左图：日期时段 176、其他 176、人群 90，三类合计占 73%。"其他"这一类比预想的大，翻回去看多是"人多的时候""第一次来的话"这类带着条件形式但没有受控类别的说法——说明这套类别表还不够用，至少缺一个"经验/次数"类。人群条件做了关键词粗分（右图）：带小孩 54、其他 29、住店客人 6、带长辈 1。粗分只做描述，不参与任何判定，因为原文写法太散——"小盆友""特种兵""恐高的姐妹"都不是标准说法，关键词兜不住。

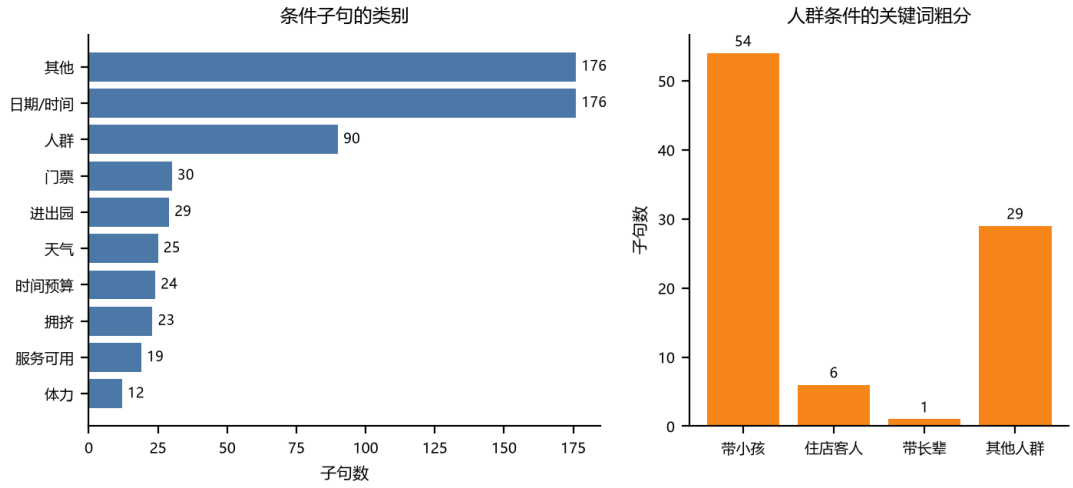


图 5-2　条件子句类别（左）与人群条件粗分（右）

表 5-5　分景区的规则数

景区	规则数	占比	备注
广州长隆	539	40.6%	官网 3 篇贡献了大部分硬性规定
珠海长隆	420	31.7%	条件类比例最高
九寨沟	368	27.7%	多为路线与时间安排
合计	1327	100%	—

还有 271 条规则被标成"条件完整"（p5_complete），占 20%。这个比例不高，主要因为大量建议本来就无条件——"早点去排队"不需要前提。把 complete 理解成"条件和结论都齐了"更准确，它不衡量建议本身有没有用。

《数据挖掘》探索性实验报告
—实验项目 6 多源程序性知识的融合与图谱可视化

6.1 输入关联与融合策略

上游四章交出来的是四种东西：清洗后的正文（P1）、实体提及（P2）、候选路线（P3）、指代关联（P4）、条件与事实（P5）。把它们接起来靠的不是文本相似度，是 ID。实体的 entity_id 由类型、景区、来源类别和规范名一起哈希，和它出现在哪篇文档、第几个字无关，所以同一句"熊猫村"在 P2、P3、P5 里指向同一个节点，中间换了多少次写法都不影响。这条设计在重跑时救过一次：P1 重清洗之后所有字符偏移全变了，但实体 ID 一个没变，只有提及位置（mention_id）跟着变。来源关联还有一条原则：不因来源类别不同而重复建立同一注册实体，也不把官方来源直接等同于当前仍有效的事实——官方写的时间同样只是历史记录。

融合顺序是固定的四步，每步都有独立的验收：先做线路清理（把 P5 里 190 个线路候选收成 29 条正式线路），再汇总时间（同一实体同一属性上的多条时刻合成一个区间并集），再汇总条件建议（同一实体的多条建议按目标合并），最后挂 LLM 主题（把一堆条目压成几条人话主题）。顺序不能换——时间汇总要读清理后的线路，主题要读汇总后的建议。

外来数据一律不新增 ID。P5 里出现过没注册的地点（"沟口""诺日朗换乘"这类），处理办法是保留原步骤文字、不建节点，也不硬并到名字相近的景点上。这样做会让一部分步骤没有 VISITS 边，但比张冠李戴强——第 3 章那条"宁可少拼一条，不要拼错一条"在这里同样适用。

6.2 对齐样例与噪声诊断

190 个线路候选里只有 29 条进了正式图。剩下的 161 条不是"错"，是"不够格当线路"：63 条只有单点（"南门进，看熊猫"），67 条是段落里的一段局部序列，24 条是纯交通片段（"坐大巴到沟口"），还有 7 条是并列备选、顺序说不清。这些全部写进 route_fragments_audit.json，每条带拒绝理由和原文证据，没有静默丢弃。

表 6-1　线路候选的判定分布

判定	条数	是否进正式图	判定依据
局部片段	67	否	跨度只覆盖段落的一部分
只有单点	63	否	不足 2 个可排序站点
完整游览序列	24	是	步骤 ≥5 且顺序明确、证据连续
交通片段	24	否	只有交通，没有游览点
无法判定	5	否	证据互相矛盾或缺失
带时刻的主题行程	3	是	带时刻分段，能看出一天怎么走
跨园区概览	2	是	原文明确写了两园先后
并列备选，顺序不明	2	否	两条以上并列，顺序无证据
合计	190	29 条	—

噪声里最麻烦的一类不是碎片，是"同名不同位置"。三篇游记都写了"熊猫馆"，但广州的熊猫馆和珠海的熊猫馆是两个地方，程序靠景区码分开了；同一景区里两个同名的"餐厅"就分不开，只能保留两条 mention、共用一个 entity_id，这是已知的欠账，体现在 P4 的"类型不符"和 P5 的"目标落不准"两处统计里。

还有一类是"父级挂错"：67 条事实的 attribute 是 parent_park，本来想表达"这个景点属于哪个园区"，但抽出来有时会变成"这个景点在园区里"，方向反了。这类没有自动修，只在质量清单里记了一笔，用的时候要回原文看一眼。

6.3 图谱模式（Schema）设计

Schema 定了 10 个节点标签和 15 种关系。定的原则只有一条：一个节点只表示一种东西，关系只表示一个方向的断言。比如线路和步骤分成两层（ROUTE 与 ROUTE_STEP），因为一条线路的属性（性质、粒度、来源）和一步的属性（到访景点、时间、交通）根本不是一个量级；合在一层的话，查"这条线路经过几个景点"要先把所有步骤属性翻一遍。

表 6-2　节点标签

标签	含义	本次节点数	主要属性
原始观察	OBSERVATION	3516	tool_name / provenance / package_key
条件规则	CONDITIONAL_RULE	1327	condition / consequence_text / force
事实	FACT	475	attribute / value_text / modality / condition
线路步骤	ROUTE_STEP	345	index / time / transport / evidence
建议汇总	ADVICE_SUMMARY	256	target_id / items / category_counts
时间汇总	TIME_SUMMARY	200	attribute / text_union / interval_unions
景点/设施	POI	179	name / role / scenic_code / advice_themes
交通路段	TRANSPORT_LEG	105	from / to / duration
交通方式	TRANSPORT	34	name / kind
线路	ROUTE	29	nature / scope / granularity / p6_curation

表 6-3　关系类型

关系	从 → 到	条数
SUPPORTED_BY	时间汇总 → 原始观察	4156
DERIVED_FROM	建议汇总 → 条件规则	1788
APPLIES_TO	规则 → 作用对象	1194
HAS_FACT	实体 → 事实	475
HAS_STEP	线路 → 步骤	345
PRECEDES	步骤 → 下一步	325
VISITS	步骤 → 景点	322
HAS_ADVICE_SUMMARY	实体 → 建议汇总	250
HAS_TIME_SUMMARY	实体 → 时间汇总	200
FROM	路段 → 起点	105
TO	路段 → 终点	105
USES_MODE	路段 → 交通方式	102
PART_OF	景点 → 上级园区	67
MENTIONS_MODE	事实 → 交通方式	34
USES_LEG	步骤 → 路段	14

属性存储有一处限制：Neo4j 的属性不能是嵌套对象或对象数组。办法是把完整负载压成 payload_json 存一份，同时把能展开的标量字段（名称、景区、类别）单独存成可查询的属性，列表里混杂类型时再拆成 xxx_json。这样既能用 Cypher 直接查，又能把原样数据取回来重算哈希——导入回读校验就是靠 payload_json。

6.4 构建实现与结构化输出

构建分四步跑，全部有幂等要求：同样的输入跑两次，产物字节相同。线路清理和四步汇总都有各自的 manifest，记着输入哈希；上游一改，下游直接拒绝运行，不会出现"用新数据配旧参数"的半新半旧结果。上游内容变化时，下游会直接拒绝运行，需显式刷新指纹后才继续。

表 6-4　构建四步与产物

步骤	产物	本次规模
线路清理	graph.json / routes.json / route_fragments_audit.json	29 条线路、161 个片段被剔除
时间汇总	entity_time_summaries.json	80 个实体、200 条汇总
条件建议汇总	condition_recommendation_summaries.json	256 组（250 组有明确目标）
LLM 主题	entity_advice_theme_summaries.json	246 组、238 个实体、525 条主题

最终图 6466 个节点、9482 条边。规模比上一轮（6763 节点）小一点，因为线路层从 197 个候选收到 29 条，步骤与相关边同步减少，差异来自线路层口径：此前片段亦计入线路。

入库已完成并回读验证：dataset 名 p6-quality-themes-v1-4d90950727ba，6466 个节点、9482 条边，按 label 与 relation 分成批量 MERGE 写入 Neo4j 5.26（本地实例，bolt://localhost:7687），随后逐节点、逐条边用 payload_json 回读比对，全部一致——reimport_no_duplicates、all_node_payloads_roundtrip、all_edge_payloads_roundtrip 均为 true，库内计数与产物一致；库中仅保留当前 dataset，旧数据清理数为 0（首次导入）。过程记录在发布快照的 validation.json，release.json 相应为 verified=true。

入库之外还做了实际查询验证：以下语句在库上执行并核对返回，查询文本与返回保存在发布快照的 cleaned_queries.json 与 live_verification.json。

表 6-6　实际查询返回与判断依据

查询	库内实际返回	判断依据
官方路线步骤（JZG-OFF-01-R…）	15 步；第 1 步 VISITS 返回 [原始森林, 沟口游客中心]	步骤数与 routes.json 一致；collect 集合是该步涉及的地点，不是步内游览顺序
POI → FACT	115 个景点、378 条事实（其余 97 条挂在路线与交通上，共 475）	与 facts 产物计数一致
CONDITIONAL_RULE	1327 条，均带 condition 与 consequence 属性	与 cleaned-v1/conditions.json 行数一致
实体主题 advice_themes_json	238 个实体带主题 JSON	与 llm_theme_unique_entity_ids 一致；JSON 内按 scenic_code 过滤后使用，通用交通方式可能同时保存多个景区内容
TRANSPORT_LEG	105 条	与 transport_legs.json 一致

主体网络视图把 ROUTE 经步骤到 POI 的访问关系投影成 ROUTE_VISITS_PROJECTED 派生边，便于观察线路覆盖与交织。它是可视化派生边，不是正式数据库新增关系，也不表示两个地点之间已确认可通达；完整的步骤、时间、条件、属性和证据仍在正式图中。



图 6-1　九寨沟清理后的完整主体网络（49 个 POI、7 条路线、63 个交通路段）

6.5 图谱可视化

全量网络只适合总览，单个实体的信息密度要看一跳邻域。以珠海"珊瑚自助餐厅"为例：两条条件规则（1.1 米以下儿童免费、早餐视客流情况逐步开放）、一条属性事实（所属横琴湾酒店）、一条时刻汇总、一条建议汇总和一条原始观察，全部直接挂在这个景点上，边上标注关系类型。这类证据网络每个景区都有离线交互版：广州长隆 124 个主体节点、203 条边，珠海长隆 84 个节点、153 条边；页面支持搜索、类型筛选、缩放平移，点击节点后在侧栏以表格与展开卡片显示基本字段、有序线路步骤、交通端点、属性、历史时间汇总、按主题整理的条件与建议以及原始证据。

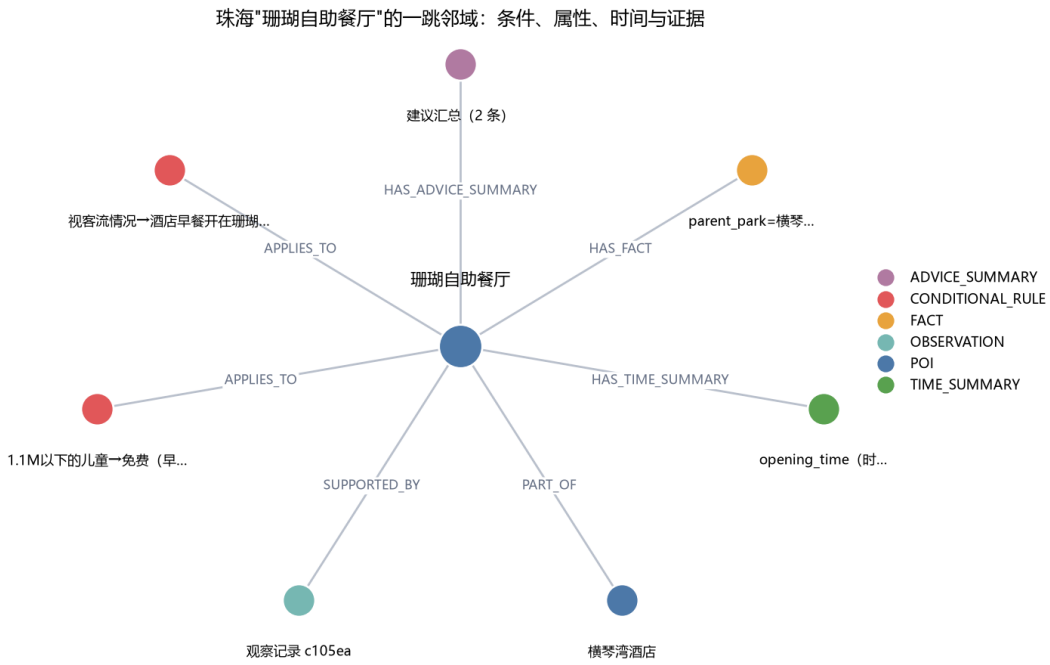


图 6-2　条件、属性与证据的一跳邻域（珠海"珊瑚自助餐厅"）

全量网络铺开是一团线（图 6-1，共 6466 个节点），静态图只画一条线路（图 6-3）：九寨沟官网那条一日游，15 步、07:00 到 17:35、途经二十来个景点。蓝色圆点是步骤，点上面的字是这一步新到的景点，点下面的时间是原文写的时刻。这张图能一眼看出两件事：路线是"一条线走到底"而不是来回折返；下午三点以后一口气串了六七个海子，该段为乘车经过，非步行。

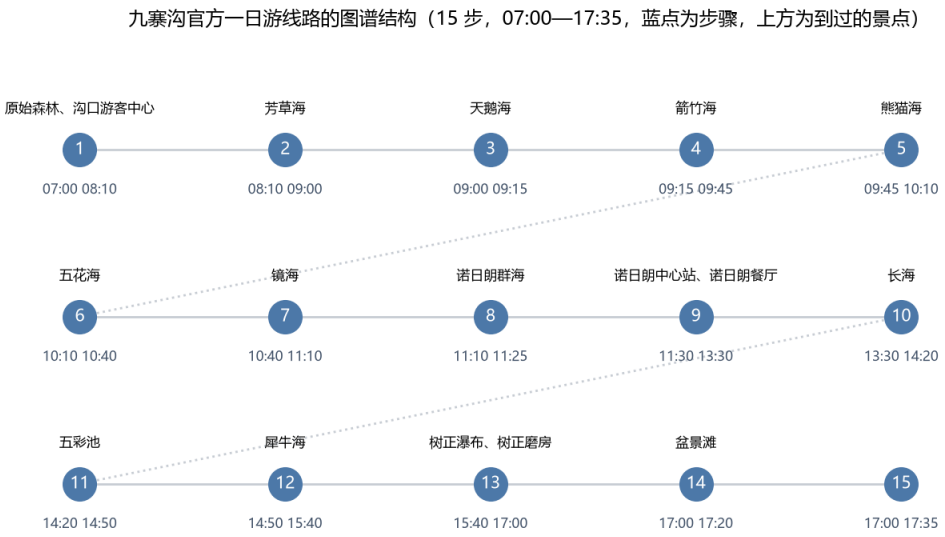


图 6-3　九寨沟官方一日线路的图谱结构（步骤 → 到访景点）

关系分布（图 6-4）说明这张图的重心在哪：SUPPORTED_BY 最多（4156 条），每条时间汇总和建议汇总都要挂回原始观察，这是"可追溯"的代价——几乎一半的边是溯源边。真正描述景区结构的边

（HAS_STEP、VISITS、PRECEDES）加起来 992 条，只占三成多。APPLIES_TO 有 1194 条，条件规则挂到对象上，这也是头重脚轻的一部分。

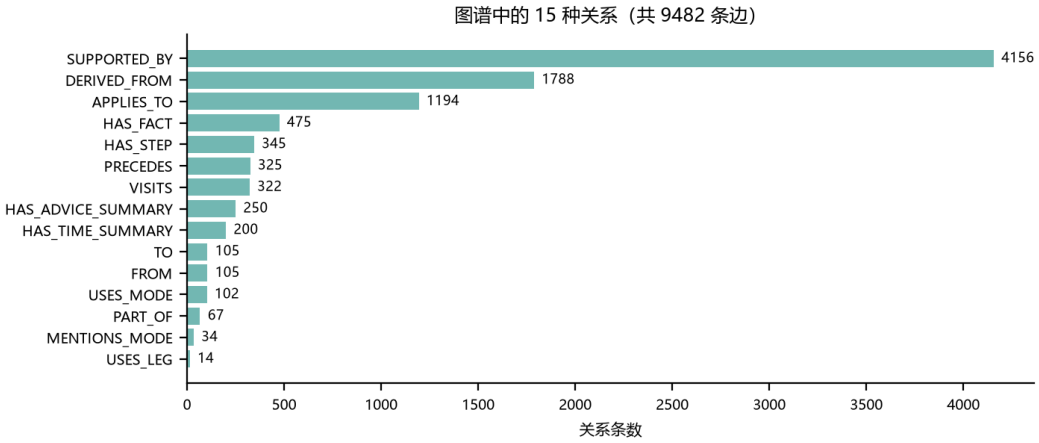


图 6-4　15 种关系的条数分布

这份东西是给人看的，不参与任何计算，也不作为提交的正式产物；正式产物是 Neo4j 中的图与 release.json。

6.6 质量验证与结果分析

质量验证分两层。结构层是硬检查：节点 ID 不重复、边不指向不存在的节点、边不重复、每条汇总都能挂回原始记录、被剔除的线路绝不出现在正式图里。这几条全是程序断言，不过就退出，没有"警告但继续"。

内容层只能抽查。时间汇总做了 200 条，覆盖 134 个实体，平均每个实体 1.5 条——这个数字很薄，因为 P5 抽到的事实一共才 475 条，很多景点根本没有时间信息。按景点本身数，179 个景点里有 99 个连一条时间汇总都没有。第 7 章排行程时经常只能给顺序、给不了时刻，根子在这里，不是 Agent 偷懒。

表 6-5　四步产物的覆盖情况

产物	数量	覆盖	说明
时间汇总	200	覆盖 134 实体；80/179 个景点	平均 1.5 条/实体，很薄
建议汇总	256	250 组有明确目标	7 组因来源是线路片段被隔离
LLM 主题	525	238 个实体、246 组	一组一调用，成功 246 组
事实	475	—	景点属性为主

主题汇总存在一处容量取舍：珠海长隆海洋王国这个实体挂了 88 个建议条目，一次调用塞不进 30K 预算。做法不是放大窗口硬塞，而是把条目二分、跑两次、再按主题标签合并。代价是跨分片的同

类主题合不起来——同一个实体可能出现两条意思接近的主题，分别来自两个分片。质量清单里记了 chunked_groups，报告里也不掩饰这一点。

三项遗留，一项刚闭环。图已导入本地 Neo4j 5.26 并逐节点、逐条边回读比对，库内计数与产物一致，release.json 的 verified 因此为 true——但库只在本机运行，未对外提供服务。主题的语义正确性没有独立复核，只有"结构和引用合法"这一层保证，每个主题还都强制以"历史资料记载，"开头，提醒读者这是语料里的说法、不是当前事实。同义主题也没有合并，语义合并这一栏写的是 false。

《数据挖掘》探索性实验报告
—实验项目 7 程序性知识应用：旅游规划Agent

7.1 架构与 LLM 接入

Agent 的结构简单到有点土：一个循环，每次问模型"下一步做什么"，模型回一个动作名加参数，程序执行、把结果塞回上下文，然后再问。没有向量检索、没有 planner-executor 分层，也没有反思链——反思写在"上一轮的观察"里，模型自己看。循环上限 6 轮，到点必须收尾。

能调的工具只有 5 个，全部是确定性函数：查景区有没有、取一条路线、搜景点、取注意事项、按人群给步骤打标。模型不能写 Cypher，也不能直接读文件，这样即使它胡说，也只会胡说在参数上，不会把库改了。

表 7-1　五个工具

工具	参数	返回	边界
check_area	area	是否存在、有几条路线、可选景区	不返回路线内容
get_route	area, days	route_id、步骤、起止时刻、是否只到园区级	一次只给一条，按天数挑
search_spots	area, visitor, top_k	景点名列表、命中总数	只取 attraction 且在景区内的
get_tips	area, keyword, visitor, limit	注意事项文本、候选条数	条件类排前面，事实类不进，人群不符的先排除
tailor_route	route, visitor	打标后的步骤（可跳过、适合孩子、适合拍照）	不删步骤，只打标

模型接的是 DeepSeek，走 OpenAI 兼容接口（OPENAI_BASE_URL 指过去，模型名 deepseek-chat）。没配 key 时自动退回规则策略：同一套工具，用 if-else 决定下一步。两条路径产出的记录格式一样，所以对比起来很直接——这一章的七场景测试用的是规则策略，7.5 节单独给一次真模型的执行过程。

保留规则策略的理由是可用性：模型接口不可用或输出不合规时，同一套工具仍能跑完全流程。七场景测试使用规则策略，结果可重复；真模型路径另行验证（7.5 节）。两者的差别在于模型会自行组合工具，也会引入规则中没有的动作。

7.2 图谱查询工具封装

工具封装的难点不在查询，在"收口"：把库里的数据结构变成模型能看懂、又不会误解的几句话。举三个具体的。

第一，景点搜索必须加两个过滤：poi_role 是 attraction、scope 是 in_area。不加的话，"成都的餐厅""路过的服务区"都会被算成九寨沟的景点，因为 P2 把全文提到的地方都标了位置。加过滤后九寨沟命中 38 个、广州 58 个，与园内实际景点数量量级相当。

第二，注意事项要排序。库内条件规则共 1327 条，直接取前三条往往是"早些准备总是好的"这类泛泛之谈。

第三，景区名要归一。用户与模型说的是"九寨沟"，库里存的是"九寨沟风景名胜区"，等值比较会直接判定为"无此景区"。check_area / get_route / search_spots / get_tips 四个入口因此统一经过别名表，"九寨沟""长隆""海洋王国"都能落到正式名上；别名表兜不住时退一步做包含匹配，仍不中就原样返回、让上层报"没有这个景区"。

```
def resolve_area(area):
    if area in AREAS: return area
    if area in AREA_ALIAS: return AREA_ALIAS[area]
    for alias, full in sorted(AREA_ALIAS.items(), key=lambda kv: -len(kv[0])):
        if alias in area: return full
    for full in AREAS:
        if full in area or area in full: return full
    return area
```

7.3 闭环编排与回退策略

闭环靠三种出口收住，缺一个就会转圈。第一种是正常收尾：信息够了，模型调 final，程序按记忆里的观察拼答案。第二种是反问：用户没说景区，或说了库外的地点，直接告诉他能排哪几个，不猜。第三种是硬边界：轮次用完、同一个调用连续失败两次、工具不存在——这三种都不许模型再试，立刻收尾并把原因写进答案。

表 7-2　出口与回退

情形	判定	输出	本次出现次数
信息齐了	模型调 final	按步骤输出行程 + 注意事项	4
用户没给景区	slots 里 areas 为空	反问，列出库里三个景区	1
景区不在库	check_area 返回不存在	说明没有数据，建议换景区	1
一天去两个景区	areas ≥2 且 days ≤1	说明排不开，建议分两天	1
轮次用完	循环计数到上限	用已拿到的路线兜底输出，注明没做裁剪	0
同一调用连错两次	失败计数 ≥2	停止重试，说明是调用问题不是数据问题	0

回退里有一条容易被忽略：模型多给的参数。它经常给 check_area 带一个 keyword，而工具只接受 area，于是每次都抛 TypeError，模型看到报错又换个说法重试，六轮全耗在这上面。现在的做法是执行前按函数签名过滤参数，多出来的丢掉并记一笔诊断，不中断、不污染答案。

步骤与天数还要对齐。容量判据有两条，取大的：原文写了完整时刻时按跨度算，一天超过 9 小时就拆（8 步横跨 12 小时同样走不动）；没有时刻时按每天 12 步算。超出容量时按实际需要的天数排，并在答案里说明是哪条判据触发的；用户要两天而库里只有一日路线时，按原文顺序拆到两天，不额外编造第二天内容。9 小时与 12 步都是启发式阈值，未按体力数据标定。

备注按集合语义保存，同一句提示只保留一条，避免在答案末尾重复出现。状态判定不完全等同于"有无备注"：只有影响结果完整性的备注才计为 partial，引擎层的诊断信息单独记录，不进入答案。

7.4 多场景功能与鲁棒性测试

七个场景是照着"能出错的类型"设计的，不是照着"能成功的类型"：两个正常需求（亲子一日、情侣两日）、一个带体量约束的（老人走不动）、一个库外地点（三亚）、一个信息不足（"帮我规划一下"）、一个自相矛盾（一天两园）、一个天数超过库内路线（九寨沟两日）。

表 7-3　七场景结果

输入	状态	轮次	步数	机器给出的理由
带爸妈去九寨沟玩一天，他们走不动太久	成行但有提醒	7	15	拿到了路线，但有需要提醒的缺口
带 6 岁孩子去广州长隆，一天怎么安排	成行	7	8	路线、裁剪、注意事项都拿到了
和女朋友去珠海长隆海洋王国玩两天	成行但有提醒	7	18	拿到了路线，但有需要提醒的缺口
去三亚玩三天	说明无数据	2	0	库里没有三亚
帮我规划一下	反问	2	0	没说去哪个景区，反问
一天想把广州长隆和珠海长隆都玩了	说明冲突	2	0	两个景区一天排不开
九寨沟玩两天，怎么安排	成行但有提醒	7	8	拿到了路线，但有需要提醒的缺口

四个成行的场景里只有一个完全干净（广州长隆亲子一日）：路线 8 步、每步有时刻、有 1 处打了"适合孩子"的标、注意事项取到 3 条。另外三个都带提醒，提醒的内容分两类：一是路线原文没写时刻，只能给顺序；二是库里没有独立的两日路线，按原文顺序把一日路线拆到两天。这两类都是数据缺口，不是逻辑错误。

三个不成行的场景都给出拒绝而非凑答案："去三亚玩三天"直接说明库内只有三个景区；"帮我规划一下"反问目的地；"一天想把广州长隆和珠海长隆都玩了"说明两园相距较远、一天无法完成，建议分两天。三个回答都简短，但都准确对应了输入的限制条件。

7.5 一次完整执行与失败分析

本案例使用真实模型执行一次，输入为"带 6 岁孩子去广州长隆，一天怎么安排"，共 5 轮、3 次工具调用，完整记录在 outputs/P7/agent_llm_demo.txt。逐轮如下：

表 7-4　真模型的一次执行

轮	模型的想法	动作	观察
0	（程序固定）拆槽位	parse_slots	景区 广州长隆 / 人群 亲子 / 1 天
1	用户指定了广州长隆、一天、带 6 岁孩子，先确认景区在库里	check_area	库里有 11 条路线
2	已确认广州长隆在库，接下来取一天的结构化路线	get_route	GZCL-NOTE-12-R…，8 步，09:30—16:30
3	已有广州长隆的一日路线（8 步，09:30—16:30），接下来按 6 岁孩子定制行程	tailor_route	成行 8 步，标注 1 处
4	已有路线和定制结果，信息足够拼一天行程，直接收尾	final	结束(注意事项由收尾层按人群补查)

与规则策略相比，模型少做了两件事：没有检索景点，也没有查注意事项，第 4 轮直接收尾。收尾层对此有兜底——注意事项按人群补查一次并记入引擎诊断，答案结构不因模型省步骤而缺块。取舍是明确的：少做不扣分，缺块才扣分。

失败案例集中在一类：参数与工具签名不一致。模型给 check_area 传 {"area": "九寨沟", "keyword": ""}，工具不认识 keyword，抛出类型错误；模型据此重试，参数不变，六轮全部消耗在同一错误上，最终状态为 no_data，并输出"路线取到了但没排出步骤"——而实际上并未取到任何路线。这类失败的成因有三层：参数未按签名过滤、景区名未归一、错误重试没有上限。对应处理分别是：执行前按函数签名裁剪参数、四个入口统一走别名表、同一调用连续失败两次即终止并由上层给出准确说明。

表 7-5　七场景：规则策略与真模型对照

场景	规则策略	真模型
带爸妈去九寨沟玩一天，他们走不动太久	成行但有提醒（7 轮）	成行但有提醒（7 轮）
带 6 岁孩子去广州长隆，一天怎么安排	成行（7 轮）	成行（5 轮）
和女朋友去珠海长隆海洋王国玩两天	成行但有提醒（7 轮）	成行但有提醒（5 轮）
去三亚玩三天	说明无数据（2 轮）	说明无数据（3 轮）
帮我规划一下	反问（2 轮）	反问（2 轮）
一天想把广州长隆和珠海长隆都玩了	说明冲突（2 轮）	说明冲突（2 轮）
九寨沟玩两天，怎么安排	成行但有提醒（7 轮）	成行但有提醒（7 轮）

对照读法：七场景的最终状态两条路径完全一致——包括反问、说明无数据、说明冲突这三个"拒绝"场景，真模型也全部正确触发。差别在过程：真模型整体轮数更少（广州一例 5 轮对 7 轮），珠海两日一例选中的是另一条 18 步的一日路线，拆天提醒由收尾层统一补齐，不依赖模型自觉。

仍存在的失败有两类。一是容量判据仍是启发式：时刻跨度 9 小时、每天 12 步都是经验值，未考虑各景区面积、爬升与停留时长，拆天只是不再硬塞，并不等于合理的两天行程。二是数据密度限制：29 条路线中仅 9 条带时刻、两日路线 1 条，两天以上的行程只能由一日路线切分而来，缺少过夜点与次日入园的语义，该限制来自语料与 P5 的抽取结果，非编排层可解决。

7.6 交互入口与配置

命令行入口是 p7_agent.py：不带参数跑七个场景并把记录写到 outputs/P7/，--ask 问一句，--llm 强制走模型决策。环境变量三个：OPENAI_API_KEY、OPENAI_BASE_URL、OPENAI_MODEL，不配就是规则模式。

```
python src/P7旅游规划Agent/p7_agent.py                    # 跑七个场景
python src/P7旅游规划Agent/p7_agent.py --ask "带老人去九寨沟一天"
```

```
export OPENAI_API_KEY=...  OPENAI_BASE_URL=https://api.deepseek.com/v1 \
       OPENAI_MODEL=deepseek-chat
python src/P7 旅游规划 Agent/p7_agent.py --llm --ask "带 6 岁孩子去广州长隆"
```

对外还有一个 Web 入口（api.py，FastAPI）。前端是 web/ 下的单页，无框架、无构建；规划走同一条 Agent 路径，接口层只做参数校验、并发与频次限制、结果整理，不复制任何业务规则。每一轮的思考与动作由 SSE 逐轮推送，前端把循环画成时间线；服务端未配置模型密钥时，接口拒绝 llm 模式并提示改用规则，前端同步禁用对应开关。公网部署有两个必配项：设置 P7_ACCESS_TOKEN 后所有接口要求口令（Bearer 头或 ?token=），前端会弹窗收集；限流与并发计数是单进程内存态，多 worker 部署需换成外置存储或保持单进程。

表 7-6　Web 接口

接口	方法	用途	返回
/api/health	GET	服务状态、数据规模、是否配置模型	JSON
/api/scenes	GET	三个景区与各自的路线数	JSON
/api/examples	GET	示例问句	JSON
/api/plan	POST	同步规划一次（限流、并发上限）	完整结果
/api/plan/stream	GET	逐轮推送思考与动作	SSE 事件流

输出三份：agent_runs.json 是七个场景的完整记录（每轮的思考、动作、观察、最终答案），agent_demo.txt 是给人读的版本，figures/p7_react.png 是循环结构和七个场景的结果图。单次 --ask 只打印、不覆盖批量记录；Web 模式不落盘，结果只随响应返回。

依赖：命令行只有 requests 和 matplotlib；Web 需要 fastapi 与 uvicorn。Agent 读的是 JSON 产物，不是数据库——图数据库在答辩现场不一定能起来，但 JSON 文件一定在。

7.7 旅游规划 Agent 示范用例

下面是广州长隆亲子一日这一例的完整答案（规则策略版本）。只把行首的①符号换成阿拉伯数字，其余一字未改：

```
按广州长隆的一日给你排了 8 步：
【上午】
  1. 09:30 开园直冲小火车（09:30）
  2. 10:15 青龙山环线（10:15）
  3. 11:30 长颈鹿广场+考拉园（11:30）
【下午】
  4. 13:30 熊猫乐园（13:30）（适合孩子）
  5. 14:30 空中缆车(选环园线)（14:30）
  6. 15:30 百虎山+火烈鸟岛（15:30）
```

```
  7. 16:30 大象传奇/南美探秘表演（16:30）
  8. 花车巡游
注意：08:50 前到北门，赶上第一波进园早、体验好很多；15:00 入园，先刷机动项目
```

这份答案能看出信息的来路。8 个步骤和时刻来自 P5 解析出的那条路线（原文是一篇游记的行程段），"适合孩子"的标来自 tailor_route 的关键词规则（熊猫、考拉、剧场、表演），两条注意事项来自 P6 汇总的条件建议（条件类优先，人群不符的已排除）。没有一句话是模型编的——这也是这一章反复强调的：Agent 的价值在编排和取舍，不在生成内容。

它也有明显的不足。第 8 步"花车巡游"没有时刻，因为原文就没写；全程没有给出每段路的交通方式和耗时；午餐没有安排（原文提到园内餐厅但没写具体时间）；"适合孩子"只标了 1 处，因为关键词表只覆盖了 4 个词。这些缺口在答案里都没有掩饰，也没有用"建议根据实际情况调整"这种话糊过去——没数据就是没数据。

《数据挖掘》探索性实验报告
—实验总结：探索性实验总结反思

一、回顾并记录实验过程中出现的至少 2 个失败现象，并深入分析可能导致这些问题的原因；尝试提出针对性的改进思路或优化方案

1、失败现象记录

失败分两类：一类在方法边界之外、应当拒答；另一类是应当做对却做错的。这一节只记录后者，并给出每一处的规模与可复核位置；前者在第 4 至 7 章的局限里说明。

表 S-1　各阶段可复现的失败与规模

阶段	失败现象	规模	可复核位置
P1	长截图 OCR 断行后与正文重复的残片仍有保留	见 1.4 节的复读行统计	辅助资料/raw_and_clean_parts.json
P3	候选区间未能构成路线（编号列表缺语境、站点不足、无实体支撑）	拒绝 258 / 扫描事件 616	candidate_inventory.json
P4	指代未消解（候选类型不符或分差不足）	394 / 545 条记录	coreferences.json
P4	单先行词消解错误	22 / 33 个目标	outputs/P4/评估/evaluation.json
P5	条件被段落边界切断，只能标为条件不完整	58 / 1327 条规则	conditions.json 的 statement_kind
P6	线路候选中的片段不能进正式图	161 个片段	route_fragments_audit.json
P7	无法给出行程（无数据、反问、冲突）	3 / 7 个场景	outputs/P7/agent_runs.json

P4 的未消解最值得拆开看：394 条里 247 条是候选池中没有类型匹配的实体，27 条是找到了候选但分差不足、判为歧义。前者属于语料问题（那句话附近确实没有地点名词），后者属于阈值问题。两类混在一起统计，只能得出"准确率低"这种没有指向的结论。

P3 的 616 个扫描事件中，接受 155 个、拒绝 258 个、标记为段落边界 201 个、重复抑制 2 个。拒绝理由集中在四类：编号列表但缺少路线语境 87 个、可排序站点不足 61 个、缺少实体识别支撑 33 个、没有完整可排序链条 16 个。边界事件指标题、日程切换、时段切换一类的位置标记，既不是候选也不是拒绝，统计时不应与候选混算。

P7 的失败全部落在"输入超出数据范围"一侧：库外景区、信息不足、跨景区一日各一例。三例都没有硬凑答案，但也没有给出降级方案（例如"仅园区级"的替代建议），这属于功能缺口，不是判定错误。

2、失败原因深度分析

把失败样本逐条归因，原因收敛到四处，且都不在"模型能力不足"这一层。

一是语料的书写方式。游记面向人阅读：一句话塞三四个景点、用箭头和表情作分隔符、时间写成"上午九十点钟"、把整篇攻略拍成图片让读者自己放大。P1 的清洗只能处理排版噪声，处理不了这种信息密度不均——同一段里可能既有五个可抽取景点，也有整段抒情。

二是行级匹配的粒度。P1 的跨来源去重按行判定，而截图文字会被 OCR 断成十几字一行，还夹着作者手写批注和识别错字（"⭐"认成"芬"）。行级精确匹配与 4-gram 覆盖率都要求"整行基本等于正文某一段"，断行之后两条判据都不成立，复读内容因此残留。这不是阈值松，是判定单位选错。

三是证据粒度。抽取记录若只挂到输入包，核对一条建议就要回原文翻；P2 的验收用例与 P4 的人工标注同样依赖裸字符偏移，上游改一处，下游整段失效。前者已由导出阶段的确定性定位解决（1622 条带引文的观察中 1448 条落到句级），标注与用例的重锚仍依赖脚本逐一处理。

四是容量与完整的矛盾。单次调用装不下整条长路线（P5 有一个包耗尽上下文预算），单个实体的建议条目也可能超过预算（P6 有 1 个实体挂 106 条建议）。放宽窗口只是推迟问题；真正的解法是切分，而切分必然损失跨片的语义合并。

3、改进/优化方案

针对第二处，把判定单位从"整行"改为"行与正文某一行的对齐"：用序列对齐计算这一行有多少字能按序对上正文，容许中间夹杂批注与错字；同时让数字过半的行（时刻表）与不足 6 字的行（地图标签）直接放行，避免误删。改后 P1 的复读行删除量由 705 行升至 769 行，重复段落残留由 12 段降至 0 段（以编辑相似度 ⩾0.8 计），语料相应减少 961 字（约 1%）。

针对第三处，两端分别处理。P5 侧把定位从模型移到程序：模型只抄原文，导出时在包内检索，精确匹配优先、规范化匹配兜底，唯一命中写 span，多处命中或找不到退回 package。本次 1791 条引文创下 1448 条句级偏移，占带引文记录的 89.3%。P2 验收用例与 P4 人工标注侧改为按整句定位、再按句内偏移还原，先行词取"位于指代之前且最近"的一处，重锚只改位置、不改内容，片段找不到直接报错。

针对第四处，P5 将超预算的包按段落切成四片重跑，合并时校验原文覆盖一致、步骤不缺；P6 将超预算实体按条目二分，两片各自校验覆盖率，合并时按主题标签去重，并把分片信息写入质量清单。两处都以可审计的切分换取完整性。

针对 P7 的接口失配，三处一并修改：调用前按函数签名过滤参数，多余参数丢弃并记录诊断；景区名做别名归一，"九寨沟"落到"九寨沟风景名胜区"；同一调用连续失败两次即终止重试，由上层给出明确说明，避免把参数错误伪装成"没有数据"。

另一项是工程性加固：所有安全闸由 assert 改为显式抛错。Python 在 -O 模式下移除 assert，而 P6 的删库前检查、图谱结构校验都依赖这些断言，改后在任何运行模式下均有效。

1、梳理现有技术方案的局限性

按当前实现，四类局限无法通过调参消除。

一是正确性缺少独立验证。P4 的 78.6% 精确率来自 5 篇已见样本，规则亦参照这批错误确定；P5 的人工核对仅覆盖 10 条；P6 的主题只验证结构与引用合法性，未复核语义；P7 的七个场景由设计者拟定。全链数字均属开发集表现，不是留出集指标。

二是覆盖率受语料密度限制。29 条路线中仅 9 条带时刻，两日路线 1 条；179 个景点中 99 个没有任何时间汇总；条件规则中"其他"类别 176 条，落不进受控类别。只能靠扩充语料或放宽证据要求解决，后者本方案不采用。

三是语义合并缺失。去重按字面执行，1327 条规则展开为 1787 个条目，由两条以上来源合并而成的仅 1 个；主题不跨实体合并，同一建议在三个景区各存一份。图谱规模因此偏大，重复表达偏多。

四是证据锚尚未贯通全链。P5 的引文已能落到句级，但带引文的记录中另有 10.7% 因改写或跨段拼接仍退回包级；P2 的验收用例与 P4 的标注仍是裸偏移，上游每改一次就要重锚一次；下游图谱节点也没有携带锚指纹，"哪条结论的依据变了"目前无法查询。

2、选定一个优先突破的技术点，并给出说明

优先突破的技术点选"证据锚"：以内容寻址的方式记录每条结论的出处，替代裸字符偏移。本次实验已完成其中第一步——P5 的引文在导出阶段定位到句，其余三步（锚结构、逐条迁移、抽查入口）仍待实现。

依据有三。其一，可核验性：报告与图谱中的结论都声称有原文依据，而依据目前是"某文件第 1234 至 1250 个字符"，文件一换即无法复核，评审无法抽查，自身也无法回归。其二，级联成本：P1 的清洗规则一旦调整，P2 至 P4 依赖偏移的标注与用例全部失效，需要逐个重锚，且仍需人工判断。其三，通用性：该改造不改变任何阶段的语义，只改变证据的记录方式，对 P4 至 P7 的接口向后兼容。

预期收益是把"文件级哈希失效"降为"条目级失效"：上游变化时，只有真正引用变动文本的条目需要重跑，其余条目可按锚迁移，既降低重跑成本，也使"哪条结论的依据变了"成为可查询的事实。

3、突破该技术点的研究思路或实施方案

方案分四步。第二步已在本次实验中实现并验证，其余三步按同一结构续接。

第一步，定义锚结构。一条证据锚含四项：引文本身（10 至 80 字）、引文前后各 20 字的上下文指纹、所在段落的内容哈希、原偏移（快速路径）。校验顺序为：按偏移切片，命中即通过；否则以引文在段落内检索，唯一命中则重锚；多处命中判为歧义；找不到则标 missing，该条结论降级为待复核，不参与下游推理。

第二步，改造 P5 的抽取协议（已实现）。抽取协议要求模型回填原文引句，导出阶段由 evidence.py 在包内定位：精确匹配优先，标点与全半角不一致时用规范化串，唯一命中记 span，多处命中或找不到退回 package。本次 1622 条带引文记录中 1448 条落到句级，留下 174 条包级记录，均为改写句或跨段拼接。定位由程序完成，模型只负责抄录，既避免算错偏移，也提高伪造引文的成本。

第三步，把级联失效从整体拒绝改为逐条迁移。清单除文件哈希外再存条目锚指纹（引文哈希）；上游变化后先执行迁移，能定位的条目保留，定位失败的标 stale，只对 stale 部分重跑。以 P1 的清洗规则调整为例，受影响的是引用被删文本的少数条目，而非整条链。

第四步，提供独立抽查入口。给定任意结论，输出其文档、段落、句子与上下文，支持人工标注″正确/错误/无法判断″。目的是把核验成本从翻查数千字降到点击一次，使人工核验样本量从 10 条提高到 100 条以上，补上 S4 的第一与第四项局限。

验收标准：以现有三个重锚脚本为基线。当前流程在偏移失效后仍需人工判断，改造后应做到零人工介入，并对每条迁移给出判定依据（偏移命中/引文唯一命中/歧义/丢失）。若锚迁移率低于当前 97.2% 的水平，则判定方案不成立，退回改进检索策略。

附录

一、提交前自查

☐ 封面及小组贡献页已填写，组内评分合计为 N×100%。
☐ 7 个实验及总结反思内容完整；未实现 / 未选做部分如实说明。
☐ 报告、代码、结构化输出、图表和日志相互对应。
☐ 总结含至少 2 个失败现象、至少 2 条具体局限及仅 1 个研究突破点。
☐ 排版规范、语句通顺、图表清晰（图有图题，表有表题，图按顺序标号、图按顺序标号）。

二、提交时间与方式（按任务书）

截止：2026 年 11 月 20 日 24:00 前。由各小组组长提交

文档命名方式：第 X 组_组长姓名_《数据挖掘》探索性实验报告

蔡毅老师班：1099235614@qq.com
许可老师班：2118151758@qq.com